CCIT Report #608
December 2006

Cluster Ranking with an Application to Mining Mailbox Networks

Ziv Bar-Yossef[*,§] Ido Guy[†,‡] Ronny Lempel[‡] Yoëlle S. Maarek[§,¶]
Vladimir Soroka[‡]

December 9, 2006

Abstract

We initiate the study of a new clustering framework, called *cluster ranking*. Rather than simply partitioning a network into clusters, a cluster ranking algorithm also orders the clusters by their *strength*. To this end, we introduce a novel strength measure for clusters—the *integrated cohesion*—which is applicable to arbitrary weighted networks.

We then present C-Rank: a new cluster ranking algorithm. Given a network with arbitrary pairwise similarity weights, C-Rank creates a list of overlapping clusters and ranks them by their integrated cohesion. We provide extensive theoretical and empirical analysis of C-Rank and show that it is likely to have high precision and recall.

A main component of C-Rank is a heuristic algorithm for finding sparse vertex separators. At the core of this algorithm is a new connection between the well known measure of vertex betweenness and multicommodity flow.

Our experiments focus on mining *mailbox networks*. A mailbox network is an egocentric social network, consisting of contacts with whom an individual exchanges email. Ties among contacts are represented by the frequency of their co–occurrence on message headers. C-Rank is well suited to mine such networks, since they are abundant with overlapping communities of highly variable strengths. We demonstrate the effectiveness of C-Rank on the Enron data set, consisting of 130 mailbox networks.

1 Introduction

Cluster ranking Clustering and community identification in networks is a vast area of study, spanning over multiple disciplines, such as computer science, sociology, physics, and biology. While the literature is abundant with diverse clustering methods, most of them fall within the same framework [20]: given a network G, find a partition of G that optimizes a predetermined objective function. The various approaches differ in the types of partitions considered (e.g., flat clustering vs. hierarchical clustering), in the assumptions made on the network (e.g., weighted vs. unweighted networks), in the choice of the objective function (e.g., k-means, normalized cuts), and in the techniques used to solve the optimization problem (e.g., agglomerative vs. divisive).

[*]Department of Electrical Engineering, Technion, Haifa 32000, Israel. Email: zivby@ee.technion.ac.il. Supported by the European Commission Marie Curie International Re-integration Grant.
[†]Department of Computer Science, Technion, Haifa 32000, Israel. Email: ido@cs.technion.ac.il.
[‡]IBM Research Lab in Haifa, Haifa 31905, Israel. Emails: {ido,rlempel,vladi}@il.ibm.com.
[§]Google, Haifa Engineering Center, Israel. Emails: {zivby,yoelle}@google.com.
[¶]Done in part when the author was at the IBM Research Lab in Haifa.

When clustering large networks, clustering algorithms frequently produce masses of clusters. This phenomenon is magnified when employing "fuzzy" or "soft" clustering methods, which partition the network into overlapping clusters. These tend to generate numerous clusters even on small networks. The abundance of clusters may make the results hard to digest and interpret. Moreover, typically only a small portion of the clusters are interesting or meaningful, giving rise to a "needle in a haystack" problem: how to select the important clusters from the masses of results returned?

In order to address the above difficulties, we propose a new clustering framework, called *cluster ranking*. Given a *cluster strength measure*, which assigns a "strength score" to every subset of nodes, and given a *maximality criterion*, which determines which sets of nodes are sufficiently "comprehensive" to be considered self-contained (rather than parts of larger clusters), a cluster ranking algorithm outputs the maximal clusters in the network, ordered by their strength. The ranking provides information that is usually not conveyed by traditional clustering: which clusters are more important than others. This information can be used, for instance, to quickly single out the most significant clusters. Similarly to search algorithms in information retrieval, cluster ranking algorithms are measured by *precision* and *recall*. Our new clustering framework is described in Section 3.

Cluster strength measure A crucial ingredient in the new framework is the choice of a suitable cluster strength measure. A proper definition of such a measure turns out to be a major challenge. Even for unweighted networks, there is no consensus on how to measure quality of a cluster or of a clustering [9, 21].

We propose a novel cluster strength measure—the *integrated cohesion*—which is applicable to arbitrary weighted networks. To define this measure, we first define the *cohesion* of unweighted clusters. Several notions of edge separators [34, 21, 16, 29, 13] have been used in the past to capture how "cohesive" an unweighted cluster is. We observe that these notions are unsatisfactory, especially in the presence of overlapping clusters. We then show that *vertex separators*, rather than edge separators, are more effective in measuring cohesion.

Extending cohesion to capture strength of weighted clusters is tricky, since edge weights have to be taken into account as well. A standard approach for handling edge weights is "thresholding": one determines a threshold T, and transforms the weighted network into an unweighted network, by keeping only the edges whose weight exceeds the threshold T. We show that standard thresholding is insufficient for measuring strength of weighted clusters. We then introduce *integrated cohesion* as an effective measure of strength for weighted clusters. The integrated cohesion of a cluster is the sum of the cohesion scores of all the unweighted clusters obtained by applying all possible thresholds to the given weighted cluster. Our new cluster strength measures are discussed in Section 4.

Cluster ranking algorithm Having set up the new framework, we present C-Rank: a cluster ranking algorithm. C-Rank is designed to work for networks with arbitrary pairwise similarity weights. The network's nodes are assumed neither to belong to a metric space nor to conform to any statistical model. C-Rank produces and ranks overlapping clusters and is thus in particular an overlapping clustering algorithm.

C-Rank works in three phases. First, it identifies a list of candidate clusters. Then, it ranks these candidates by their integrated cohesion. Finally, it eliminates redundant clusters—ones that are non-maximal.

At the core of C-Rank is a hierarchical overlapping clustering procedure, which constructs a hierarchy of overlapping clusters in unweighted networks. This procedure may be of independent interest. Given a network G, the procedure finds a *sparse vertex separator* in G, and uses the

separator to split the network into a collection of overlapping clusters. The procedure then recurses on each of the clusters, until reaching cliques or singletons. Interestingly, the hierarchy produced by the procedure may be a DAG (Directed Acyclic Graph), rather than a tree. We provide rigorous theoretical analysis of this procedure and show that it is guaranteed to find *all* maximal clusters in G (note that other soft clustering algorithms may not have this guarantee and are thus less useful in our framework). The procedure may run in exponential time in the worst-case—an unavoidable artifact of the quest for overlapping clusters. Yet, we show that its running time is only polynomial in the output length. In practice, it took C-Rank several minutes to cluster networks consisting of more than a thousand nodes on a standard laptop PC.

Given a weighted network, C-Rank produces candidate clusters by transforming the network into multiple unweighted networks using a gradually increasing threshold. The hierarchical overlapping clustering procedure is used to extract clusters from each of these unweighted networks. Full details of the algorithm are given in Section 5.

Finding sparse vertex separators A fundamental ingredient of the precedure for building a hierarchy of clusters in unweighted networks is the detection of sparse vertex separators. In the general case, the problem is NP-hard [7], yet some approximation algorithms exist for finding sparse vertex separators, while making use of semi-definite programming [25, 10]. As solving a semi-definite programming problem can be very computationally expensive, these algorithms are not always practical. We thus propose a different heuristic method for finding sparse vertex separators, which is based on multicommodity flow.

In order to find a sparse vertex separator, we show a connection between sparse vertex separators and vertex congestion in multicommodity flow: the congestion on the most congested node under a multicommodity flow provides an approximation of the sparsest vertex separator. The proof is based on an extension of an argument that appears in the work of Leighton and Rao [25]. This result leads to a heuristic algorithm for finding a sparse vertex separator by iteratively removing the most congested nodes from the network until the network decomposes into two or more components.

Finding the optimal multicommodity flow requires solving a linear programming problem, which is computationally expensive. We thus find instead the most congested nodes under a specific multicommodity flow — the shortest paths multicommodity flow. It was empirically shown that for many networks the value of this flow is close to the value of the optimal multicommodity flow [29]. The advantage of using a shortest path flow is that the most congested nodes can be found using a dynamic programming algorithm, which is relatively efficient. We present such an algorithm, which is based on an algorithm by Girvan and Newman [16] for finding congested edges under the same flow. As part of building the algorithm, we point out a connection, which, to the best of our knowledge, has not been known before, between edge and vertex betweenness and the congestion on edges and nodes in a multicommodity flow. This connection may be of independent interest. Section 6 describes in detail our method for finding sparse vertex separators.

Mailbox networks We demonstrate the efficacy of the novel framework and of the C-Rank algorithm in a new domain: clustering mailbox networks. A *mailbox network* is an "egocentric" social network [32, 39]—a network centered around a root individual. Unlike global "sociocentric" networks [15], it provides the subjective viewpoint of an individual on her social environment. A mailbox network is generated by mining messages in an individual's mailbox. Actors in this network are the individual's group of contacts. The weight of an edge connecting two actors is the number of messages on whose header both actors appear (either as co-recipients, or as a sender and a recipient). This weight represents the strength of the ties between the two actors from the

individual's perspective. Mailbox networks are typically abundant with overlapping communities of variable strengths, and thus C-Rank is highly suitable for them.

Automatically discovering communities within mailbox networks could be beneficial in various applications. In email and personal information management systems, the knowledge of one's favorite communities could support the automation of a variety of features such as automatic completion of groups when entering multiple recipients, detection of missing or redundant recipients, suggestion of additional contacts given a certain topic, etc. Email communities might also help in spam filtering by identifying "spam groups" [6] and possibly in enhancing the management of blocked lists. In the intelligence domain, communities can evidence gangs or potential criminal groups around known criminals.

Experimental results We evaluated C-Rank on our own mailboxes as well as on about 130 mailbox networks generated from the Enron data set [23]. To evaluate the quality of C-Rank, we adapted the popular edge betweenness clustering algorithm of Girvan and Newman [16] to the cluster ranking framework, and compared the two algorithms. We found that C-Rank dominates the edge betweenness algorithm under almost any metric. We also evaluated the robustness of C-Rank under random removal of data, and found it to be quite resilient. These results are presented in Section 7.

2 Related work

The literature on clustering and community detection consists of numerous measures of quality for communities and clustering. These vary from distance-based metrics (such as minimum diameter, sum-of-squares, k-means, and k-medians, cf. [19]), to graph-theoretic measures (such as normalized cuts [34], conductance [21], degree-based methods [13, 14, 18], performance [38], edge betweenness [16], modularity [29], bipartite cores [24], and k-cliques [30]), to statistical methods (e.g., [3]). Unfortunately, there is no single, widely acceptable, definition, and many of the above notions are known to work badly in some situations (cf. [9, 21, 22]). Furthermore, many of the above measures are suited for restricted scenarios, such as hard partitional clustering, model-based clustering, or clustering of metric space data points.

Fuzzy cluster analysis (cf. [17]) is a branch of data clustering, in which each data point can be associated with multiple clusters with different confidence probabilities. Fuzzy clustering can be used in particular to generate overlapping clusters. Nevertheless, most of the classical work in the area (e.g., Fuzzy c-means) assumes the data points lie in a metric space, which respects the triangle inequality. We consider arbitrary weighted networks whose induced distance measure does not necessarily satisfy the triangle inequality. More recent studies (e.g., [36, 33, 2, 8, 30, 4, 5]) address the general scenario of networks with arbitrary pairwise similarity weights. These algorithms substantially differ from ours, because they do not rank clusters and are not guaranteed to output all maximal clusters.

Pereira, Tishby, and Lee [31] present a hierarchical soft clustering algorithm for weighted networks that lie in a metric space, using a technique called *deterministic annealing*. This technique bares some similarity to the increasing threshold used by C-Rank to find candidate clusters.

Several works studied communities in email networks. Tyler *et al.* [37] mined communities in *sociocentric* email networks, i.e., ones extracted from the viewpoint of an organization's mail server. Fisher and Dourish [12, 11] study egocentric mailbox networks as we do, yet they detect communities by manual inspection and not by an automatic algorithm. Boykin and Roychowdhury

[6] mined communities in mailbox networks in order to detect "spam communities". Their clustering algorithm, however, is too coarse to reveal the overall community structure of the network. McCallum *el al.* [26] cluster email messages in an individual's mailbox, based on their text content, rather than on the message headers.

3 Cluster ranking framework

Throughout, $G = (V_G, E_G)$ is an undirected network and $n = |V_G|$ is the number of nodes in G. G has no parallel edges, yet self loop edges are allowed. Every edge $e \in E_G$ is associated with a non-negative weight $W(e)$. The weight represents the strength of the tie between the two connected nodes. The self loop weight represents the intrinsic "importance" of the corresponding node. If u and v are not connected by an edge, we implicitly assume $W(u, v) = 0$. In the special case all edge weights are 1, G is called an *unweighted* network. Note that edge weights can be arbitrary, and in particular need not correspond to a metric.

The first basic ingredient of the cluster ranking framework is the following:

Definition 1 (Cluster strength measure). A *cluster strength measure* is a function μ, mapping networks to non-negative real values. $\mu(C)$ is the *cluster strength* of a network C.

Intuitively, $\mu(C)$ represents how "strong" C is as a cluster. There could be many possible realizations of this definition, depending on the properties of a cluster viewed as making it "strong". One simple example is the *clique strength measure* for unweighted networks. This measure takes on only Boolean values: a network C is of strength 1 if it is a clique, and is of strength 0 otherwise. Under this measure, then, only cliques are considered clusters. In Section 4 we propose a new strength measure that suits weighted networks with overlapping clusters.

Cluster strength is an intrinsic property of the network C. Typically, C is a subset of a larger network G. The cluster strength depends only on the internal connectivity within C and not on how C is connected to the rest of the network G. Nevertheless, cluster strength by itself is clearly insufficient to represent the "desired" clusters in a network. For example, a small clique A, which is embedded in a larger clique B, is strong under the clique strength measure, but is evidently not very interesting, because it is simply an integral part of the larger clique. In order to capture these redundant clusters, we introduce the second basic ingredient of the framework:

Definition 2 (Maximality criterion). Let $G = (V_G, E_G)$ be a network. A *maximality criterion* is a Boolean function, mapping subsets of V_G to $\{0, 1\}$. All the subsets that are mapped to 1 are called *maximal* and all the subsets that are mapped to 0 are called *non-maximal*.

A natural maximality criterion in the cliques example maps a set C to 1 if and only if it is a clique and not contained in any other clique. The maximal clusters in this case are the maximal cliques in G.

We can now state the cluster ranking problem:

The cluster ranking problem

Input: *A network G.*
Output: *The maximal clusters in G ordered by their strength.*

The cluster ranking problem, as stated, could be a hard optimization problem. One immediate difficulty is that the number of maximal clusters may be very large, so just outputting them may take a long time. We thus measure the performance of ranking algorithms not only relative to the input length but also relative to the output length (the so called "input-output complexity"). A more serious problem is that typically the computational problem itself (even when the output is short) is hard. Sometimes even the computation of $\mu(C)$ on a given network C might be intractable. It follows that in reality we cannot expect a ranking algorithm to provide an exact solution to the ranking problem. A typical ranking algorithm may include on its list non-maximal clusters and/or may miss some maximal clusters. We thus adapt information retrieval metrics to evaluate the quality of cluster ranking algorithms.

For a network G and for a ranking algorithm A, let $A(G)$ be the list of clusters returned by A when given G as input. Let $I(G)$ denote the "ideal" desired output of A, i.e., the list of all maximal clusters in G. Our metrics try to quantify the difference between $A(G)$ and $I(G)$.

The *recall* of A is the fraction of maximal clusters output by A:

$$\text{recall}(A, G) = \frac{|A(G) \cap I(G)|}{|I(G)|}.$$

The *precision* of A is the fraction of maximal clusters among the clusters output by A:

$$\text{precision}(A, G) = \frac{|A(G) \cap I(G)|}{|A(G)|}.$$

4 New cluster strength measure and maximality criterion

In this section we develop a new measure of cluster strength and a corresponding maximality criterion. Our measure is quite general, and in particular is suited for finding overlapping clusters in networks with arbitrary weights.

4.1 Unweighted networks

We start with the simpler case of unweighted networks. In such networks, two nodes are "satisfied" being in the same cluster if and only if they are connected by an edge. Satisfied pairs of nodes wish to keep the cluster intact, while unsatisfied pairs want to break it apart. A strong cluster is one which is "cohesive" in the sense that no decomposition of the cluster into pieces will create more satisfaction than keeping the cluster undivided.

The above intuition has been formalized via various notions of graph partitioning, such as normalized cuts [34], conductance [21], edge betweenness [16], modularity [29], and relative neighborhoods [13]. The underlying principle in all these approaches is the same: a network is cohesive if and only if it does not have any "weak" edge separator (a.k.a. edge cut). An *edge separator* is a subset of the network's edges whose removal from the network makes the network disconnected. The above approaches differ in the way they measure the "weakness" of the edge separator.

We observe that regardless of the weakness measure used, edge separators sometimes fail to capture the cohesion of networks, especially in the presence of overlapping clusters. While the existence of a weak edge separator in a network is sufficient to make the network noncohesive, it is not a necessary condition. A simple example for this is illustrated in Figure 1. Here, we have two cliques of size n that overlap in a single node. It is easy to check that any edge separator of this network has $\Omega(n)$ edges and thus will be considered relatively strong almost under any

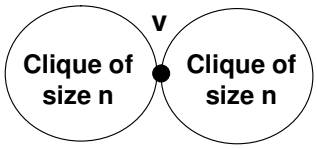


Figure 1: Overlapping cliques.

measure. However, this network is clearly noncohesive because it naturally decomposes into the two overlapping cliques.

We propose using *vertex separators*, rather than edge separators, to measure the cohesion of a network. A vertex separator is a subset of the network's nodes whose removal leaves the network disconnected. In the example network above, the single node in which the two cliques overlap is a vertex separator. Intuitively, a network is cohesive if and only if it is robust to removal of nodes, i.e., it does not have a small vertex separator that separates the network into large pieces.

Formally, a *vertex separator* of an undirected and unweighted network $C = (V_C, E_C)$ is a subset S of V_C s.t. the network induced on $V_C \setminus S$ (i.e., the network obtained from C by removing S and all its incident edges) is disconnected. A *partition* induced by a vertex separator S is a partition of $V_C \setminus S$ into two disjoint sets A and B s.t. no edge in E_C connects A and B. Note that the same separator may induce multiple different partitions. We define the cohesion of a network via the notion of "vertex separator sparsity" (cf. [1, 10]):

Definition 3 (Network cohesion). Let $C = (V_C, E_C)$ be an unweighted network. The *cohesion* of C is:

$$\text{cohesion}(C) = \min_{(S,A,B)} \frac{|S|}{\min\{|A|, |B|\} + |S|},$$

where the minimum is over all vertex separators S of C and over all partitions of C induced by S. The cohesion of a singleton (a network of size 1) is 1, if it has a self loop, and 0 otherwise.

The ratio $\frac{|S|}{\min\{|A|,|B|\}+|S|}$ is called the *sparsity* of the partition. It is minimized when S is small and A and B are both large. That is, under the above definition, a network C is cohesive if and only if it cannot be broken into large pieces by removing a small number of nodes from the network. The fact that the two pieces are large is important, because it may be easy to cut off a small part from a network, even if the network is cohesive, e.g., by isolating a single leaf node.

The cohesion of a network takes on values between 0 (for disconnected networks) and 1 (for cliques). Note that sparse vertex separators subsume weak edge separators: if the network has a weak edge separator, then it must also have a sparse vertex separator. However, as the example network above demonstrates, the converse is not true.

Computing the cohesion of a network is an NP-hard optimization problem [7]. Yet, it can be approximated in polynomial time [25, 10]. In this paper we use a faster heuristic flow-based approximation of network cohesion, which is described in Section 6.

4.2 Weighted networks

In weighted networks cohesion is no longer the sole factor determining cluster strength. Edge weights should be taken into account as well. For example, a clique of size n all of whose edges are of weight 1 and a clique of size n all of whose edges are of weight 100 are equally cohesive. Yet, clearly the latter clique is "stronger" than the former. How do we then combine cohesion and edge weights into a single strength measure?

One of the popular methods for dealing with weighted networks is "thresholding" (see, e.g., [30]): given a weighted network C, one selects a *weight threshold* $T \geq 0$, and transforms C into an unweighted network C^T by changing all the weights that are greater than T to 1 and all the weights that are at most T to 0. C is then clustered by simply clustering C^T. This approach, though, is too coarse, especially in the presence of overlapping clusters. To illustrate the problem, consider the example network depicted in Figure 2. In this example, we have two nested cliques. A smaller clique A all of whose edges are of weight 10 is nested within a larger clique B, whose other edges are of weight 1. Clearly, both A and B are clusters of interest, yet any choice of a single threshold results in the loss of at least one of them. If the threshold is set to be less than 1, then A is lost, while if the threshold is set to be at least 1, then B is lost.

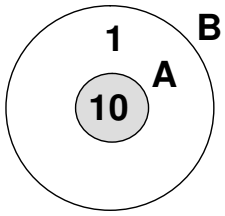


Figure 2: Nested cliques.

Our crucial observation is that in order to determine the strength of a weighted network, we should not fix a single weight threshold, but rather consider all possible weight thresholds *simultaneously*. A strong cluster is one that has high cohesion under many different thresholds. Formally, this is captured by the following measure:

Definition 4 (Integrated network cohesion). Let C be a weighted network. The *integrated cohesion* of C is defined as:

$$\text{intcohesion}(C) = \int_0^\infty \text{cohesion}(C^T)dT.$$

For example, the integrated cohesion of a clique all of whose edges are of weight k is k. Similarly, the integrated cohesion of a singleton whose self loop weight is k is also k. Although integrated cohesion is defined as a continuous infinite sum, in practice: (1) It is always finite, as for all thresholds T that are greater than the maximum edge weight, C^T is an empty graph, and thus $\text{cohesion}(C^T) = 0$. (2) It can be computed by summing up a finite number of cohesion values. The only weight thresholds in which the induced unweighted network can change are the distinct edge weights of C. Therefore, by summing up at most $|E_C|$ cohesion scores, one can compute the integrated cohesion.

4.3 Maximality criteria

We now define maximality criteria for weighted and unweighted networks.

Unweighted networks In order to define maximality in unweighted networks, we first discuss the notion of *cluster subsumption*. Our maximal clusters will be the ones that are not subsumed by any other cluster.

Let us begin with a motivating example. Consider the two clusters depicted in Figure 3. The larger cluster D is the union of two overlapping cliques D_1, D_2 of size n each, whose overlap is of

size k. The smaller cluster C is a union of two overlapping cliques C_1, C_2 of size $n/2$ each. C_1 is a subset of D_1, C_2 is a subset of D_2, and the overlap between C_1 and C_2 coincides with the overlap between D_1 and D_2. It can be checked that C is more cohesive than D, yet clearly C is "uninteresting", since it is an integral part of D. We would like then to say that D *subsumes* C, and thus C cannot be maximal. In fact, in this example C is not unique. Any union of a subset of D_1 with a subset of D_2 whose overlap coincides with $D_1 \cap D_2$ will give a cluster, which is more cohesive than D, but is subsumed by D.

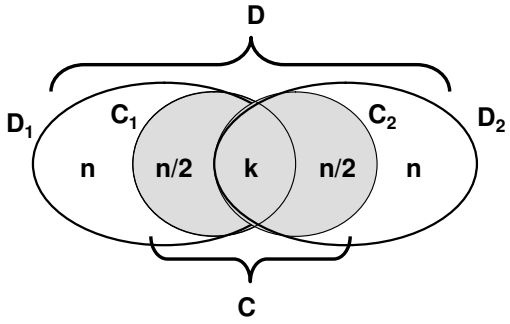


Figure 3: Example of cluster subsumption.

What really makes D subsume C in the above example? If we break up D into its natural clusters D_1 and D_2, then also C breaks up into different pieces (C_1 and C_2). That is, the partition of D *induces* a partition of C. In fact, it can be argued that every time a partition of some set A into clusters induces a partition of a subset $B \subseteq A$ into clusters, then A subsumes B. In order to formally define subsumption, we introduce some terminology:

Definition 5 (Covers). Let V be a set. A *cover* of V is a collection of subsets $V_1, \ldots, V_k \subseteq V$ whose union is V: $\bigcup_{i=1}^{k} V_i = V$. Note that sets participating in a cover, unlike a partition, can overlap. The cover is called *trivial*, if at least one of $V_1, \ldots, V_k$ equals V. Given a subset $V' \subseteq V$, the cover of V' *induced* by $V_1, \ldots, V_k$ is $V'_1 = V_1 \cap V', \ldots, V'_k = V_k \cap V'$.

Vertex separators not only provide us with a robust notion of network cohesion, but they also enable us to break up networks into their "natural" top-level clusters:

Definition 6 (Vertex separator cover). Let $G = (V_G, E_G)$ be an unweighted network and let S be a vertex separator of G. Let $A_1, \ldots, A_k$ be the k connected components of $G \setminus S$. The *S-cover* of G is $S \cup A_1, S \cup A_2, \ldots, S \cup A_k$.

Note that the clusters participating in a vertex separator cover overlap, because all of them contain the separator. In the example depicted in Figure 3, the intersection $D_1 \cap D_2$ is a (sparsest) vertex separator of both D and C. The corresponding covers of D and C are D_1, D_2 and C_1, C_2, respectively. We can now define subsumption:

Definition 7 (Subsumption). Let $C \subsetneq D$ be two clusters in an unweighted network G. D is said to *subsume* C, if there exists a sparsest vertex separator S of D, whose corresponding cover induces a non-trivial cover of C.

In the example above D subsumes C, because the cover corresponding to the sparsest vertex separator of D is D_1, D_2, and this cover induces the non-trivial cover C_1, C_2 of C.

Two remarks are in order:

1. The notion of subsumption does not properly handle cliques, because the vertex separator of any clique is already trivial, and thus the covers it induces on all its subsets are trivial too. In particular, non-maximal cliques are not subsumed by any of their supersets under this definition. To fix this anomaly, we explicitly postulate that if D is a clique, then it subsumes all its proper subsets.

2. In the current definition of subsumption we consider only the sparsest separators of D. But what happens if D has a very sparse separator, which is only *close* to being the sparsest, and this separator induces a non-trivial cover of C? Clearly, D should be considered as subsuming C also in this case. We thus define α-subsumption as follows: D *α-subsumes* C, if D has a vertex separator whose sparsity is at most $(1+\alpha)$ times the optimum, and whose corresponding cover induces a non-trivial cover of C. α is called the *subsumption parameter*.

We can now define maximality in unweighted networks:

Definition 8 (Maximality in unweighted networks). Let $G = (V_G, E_G)$ be an unweighted network. A subset $C \subseteq V_G$ is called *maximal*, if it is not subsumed by any other subset of V_G.

In the example network depicted in Figure 3, the cluster C cannot be maximal, because it is subsumed by the cluster D.

Using α-subsumption rather than subsumption, we could define a corresponding notion of *α-maximality*. For simplicity of exposition, our presentation focuses on standard maximality ($\alpha = 0$).

We would like to show that the above maximality criterion captures natural types of clusters:

Lemma 9. *Let G be an unweighted network. Then, the connected components of G and the maximal cliques in G are maximal.*

Proof. We start with the connected components. Let C be a connected component of G. If G itself is connected, then $C = G$, and the whole network is always maximal, because it is simply not a proper subset of any other subset of G. Suppose, then, that G is disconnected. Let D be any subset of G that strictly contains C. Since C is a connected component, D must be disconnected. It follows that the sparsest vertex separator of D is empty and the cover corresponding to this separator consists of the connected components of D. C must be one of these components, and thus the cover induced on C by the vertex separator cover is trivial. We conclude that D does not subsume C. As this holds for every D that strictly contains C, C is maximal.

Let C be now a maximal clique in G. If $C = G$, we are done, so suppose C is a proper subset of G. Let D be any subset of G that strictly contains C. D cannot be a clique, because C is a maximal clique. Let S be the sparsest vertex separator of D, let $A_1, \ldots, A_k$ be the connected components of $D \setminus S$, and let $D_1 = S \cup A_1, \ldots, D_k = S \cup A_k$ be a vertex separator cover of D. Since D is not a clique, then $k \geq 2$. Let $C_1 = C \cap D_1, \ldots, C_k = C \cap D_k$ be the cover of C induced by $D_1, \ldots, D_k$. We next argue that there must be an $i \in \{1, \ldots, k\}$ s.t. $C \subseteq D_i$.

Suppose, to reach a contradiction, that C is not a subset of any of $D_1, \ldots, D_k$. It follows that C must intersect at least two sets among $A_1, \ldots, A_k$. Suppose, for example, $C \cap A_1 \neq \emptyset$ and $C \cap A_2 \neq \emptyset$. Let u be any node in $C \cap A_1$ and let v be any node in $C \cap A_2$. Since A_1, A_2 are different connected components of $D \setminus S$, then u and v are not connected by an edge in G. On the other hand, both u and v belong to C, contradicting the fact C is a clique.

We conclude that $C \subseteq D_i$ for some i. Hence, $C_i = C$, implying $C_1, \ldots, C_k$ is a trivial cover of C, and thus C is not subsumed by D. Since this holds for all subsets D of G that strictly contain C, C is maximal. □

Weighted networks Having defined maximality in unweighted networks, it is quite straightforward to extend the definition to weighted networks:

Definition 10 (Maximality in weighted networks). Let $G = (V_G, E_G)$ be a weighted network. A subset $C \subseteq V_G$ is called *maximal*, if there exists at least one threshold $T \geq 0$, for which C is maximal in the unweighted network G^T.

In the example network depicted in Figure 3, if the edges of the cluster C are all of weight 10 and the rest of the edges in the cluster D are of weight 1, then C is now maximal, because it is maximal in the unweighted network G^T, for all $T \in [1, 10)$.

Remark. A common pattern in social networks is the "onion pattern" [12]: a sequence of nested clusters, each of which is only slightly stronger than the cluster it is contained in. This pattern characterizes, for instance, the collaboration within projects: most of the interaction occurs within a core team of project members, while larger circles of consultants are only peripherally involved. The different layers of an "onion" give rise to clusters that are all maximal. Nevertheless, it is clear that not all of them are of interest. This motivates us to search for clusters that are not just maximal but are rather maximal *by a margin*.

We say that a cluster C is *maximal by a margin* ϵ, if there exists an interval $[T_1, T_2]$, where $T_2 \geq (1+\epsilon)T_1$, s.t. C is maximal in G^T, for all $T \in [T_1, T_2]$. For instance, if in the network depicted in Figure 3, the weight of edges in C is 1.1 rather than 10, then C is maximal in G^T, for $T \in [1, 1.1)$. It follows that C is maximal, if the maximality margin ϵ is at most 0.1, but is not maximal, if this margin is greater than 0.1.

5 The C-Rank algorithm

In this section we describe C-Rank: an algorithm for detecting and ranking clusters in weighted networks. C-Rank consists of three major phases: (1) identification of candidate clusters; (2) ranking the candidates by integrated cohesion; and (3) elimination of non-maximal clusters.

5.1 Candidate identification in unweighted networks

We start with the description of a candidate identification algorithm for unweighted networks. In Section 5.2, we build on this algorithm to identify candidate clusters in weighted networks. Our candidate identification procedure (see Figure 4) finds the sparsest vertex separator of the given network, uses its induced cover to split the network into overlapping clusters, and then recurses on the clusters. The recursion stops when reaching cliques or singletons, since they cannot be further partitioned. If more than one vertex separator exists, one of them is chosen arbitrarily.

As the procedure detects overlapping clusters, it may encounter the same cluster more than once. For example, if clusters C and C' overlap, then a cluster within the intersection $C \cap C'$ may be detected both at the recursive call on C and at the recursive call on C'. Our procedure therefore checks that a cluster is not already on the list, before recursively processing it. This guarantees that the procedure does not perform redundant work.

The procedure not only produces a list of maximal clusters from the given network G, but also implicitly organizes them in a *hierarchy*, similarly to hierarchical clustering. The difference is that here, due to the overlapping clusters, the hierarchy is not necessarily a tree, but is rather a DAG (directed acyclic graph). The root of the hierarchy is the whole network G and its leaves are either singletons or cliques. Each cluster in the hierarchy is covered by its child clusters. We call such a hierarchy a *hierarchical overlapping clustering*.

1:**Procedure** unweightedCRank(G, $\mathcal{L}$)
2: add G to $\mathcal{L}$
3: if G is a clique or a singleton return
4: $S :=$ sparsest vertex separator of G
5: $A_1, \ldots, A_k :=$ connected components of $G \setminus S$
6: for i = 1 to k do
7: $G_i :=$ sub-network of G induced on $S \cup A_i$
8: if G_i not already in $\mathcal{L}$ then
9: unweightedCRank(G_i, $\mathcal{L}$)

Figure 4: Identifying candidate clusters in unweighted networks.

Example run Figure 5 shows an example run of the above procedure on a simple 5-node network. The procedure first detects $S = \{c, d\}$ as the sparsest vertex separator of the network and removes it from the network. The resulting connected components are $A_1 = \{a, b\}$ and $A_2 = \{e\}$. The procedure adds S to each of the connected components, obtaining the two overlapping clusters $\{a, b, c, d\}$ and $\{c, d, e\}$. No recursive calls need to be made in this example, because both of these clusters are cliques.

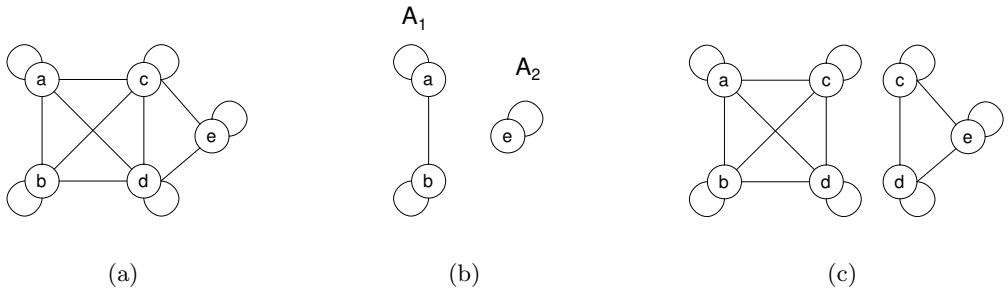


Figure 5: Identifying unweighted clusters: Example run.

Analysis We next analyze the quality and the performance of the algorithm. We start by showing that the algorithm is guaranteed to have an ultimate recall of 1:

Lemma 11. *Given an unweighted network G, C-Rank outputs all the maximal clusters in G.*

Proof. Suppose, to reach a contradiction, there exists a maximal cluster C, which is not output by C-Rank. That is, C does not belong to the hierarchy of overlapping clusters constructed by the algorithm. C is clearly a subset of the root of the hierarchy (the whole network G). It cannot be a subset of any of the leaves of the hierarchy, because each of these leaves is either a singleton (which has no proper subsets) or a clique (all of whose proper subsets are non-maximal). We conclude that there must be some internal node D in the hierarchy s.t. $C \subseteq D$ but C is not contained in any of the children of D.

Let $D_1, \ldots, D_k$ be the children of D in the hierarchy. $D_1, \ldots, D_k$ form a vertex separator cover of D. Let $C_1 = C \cap D_1, \ldots, C_k = C \cap D_k$ be the cover of C induced by $D_1, \ldots, D_k$. Since C is not a subset of any of $D_1, \ldots, D_k$, then none of $C_1, \ldots, C_k$ equals C. We conclude that $C_1, \ldots, C_k$ is a

non-trivial cover of C, and thus C is subsumed by D. This contradicts our assumption that C is maximal. □

The analysis above establishes that C-Rank has ultimate recall. But what about precision? How likely is C-Rank to output clusters that are non-maximal? When C-Rank splits a cluster C into sub-clusters $C_1, \ldots, C_k$ using a vertex separator, $C_1, \ldots, C_k$ are not subsumed by C. If C is maximal, then $C_1, \ldots, C_k$ are likely to be maximal too. However, this intuition does not always work, as $C_1, \ldots, C_k$ may be subsumed by subsets of C. This situation, though, rarely happens. We do not have theoretical guarantees about the precision of C-Rank, but we provide empirical evidence in Section 7 that its precision is reasonable.

The performance of C-Rank is directly related to the number of clusters it produces. Clearly, since the number of maximal clusters can be exponential in the size of the input network G, then C-Rank may run for an exponential amount of time. However, when the list of maximal clusters is short, then C-Rank will also run more quickly:

Lemma 12. *Suppose that on a given network G C-Rank outputs a list of m candidate clusters $C_1, \ldots, C_m$. Then, the running time of C-Rank is $O(\sum_{i=1}^{m} (r(|C_i|) + |C_i|^2))$, where $r(n)$ is the amount of time needed to compute the sparsest vertex separator of a network of size n.*

Proof. For each cluster C_i output by C-Rank, C-Rank performs the following tasks: (1) testing whether C_i is a clique or a singleton ($O(|C_i|^2)$ time); (2) computing a sparsest vertex separator S of C_i ($O(r(|C_i|))$ time); (3) computing the connected components of $C_i \setminus S$ ($O(|C_i|^2)$ time using BFS); (4) iterating over the connected components of $C_i \setminus S$ and checking whether each one is already on the list of candidates $\mathcal{L}$ (a total of $O(|C_i|)$ time). Hence, the total running time for each cluster C_i is $O(r(|C_i|) + |C_i|^2)$. □

Recall that finding the sparsest vertex separator of a network is NP-hard. Hence, in a naive implementation of C-Rank, $r(n)$ will be exponential in n, which is of course unacceptable. Therefore, C-Rank does not compute exact sparsest separators, but rather approximate sparsest separators. These separators are computable in $O(n^4)$ time. The approximation procedure is described in Section 6.

5.2 Candidate identification in weighted networks

The simplest way to extract all maximal clusters from a weighted network $G = (V_G, E_G)$ is the following. We enumerate all possible thresholds T (there are at most $|E_G|$ such thresholds), compute G^T, and output all the maximal clusters in G^T using the procedure unweightedCRank. This guarantees that we output all maximal clusters of G, and hence obtain ultimate recall.

The above brute force enumeration could be very time-consuming, since we need to make up to $|E_G|$ calls to unweightedCRank, and each call is made over the entire network G. Furthermore, this approach tends to be wasteful, as we may identify the same clusters again and again under different thresholds. For example, a maximal clique all of whose edges are of weight T will be discovered at all thresholds $T' < T$. A natural question is then whether we can trade the ultimate recall guarantee for better performance?

To this end, we make the following observation. What is the reason for a cluster C to be maximal at G^T, for some threshold T, while not being maximal at $G^{T'}$, for all $T' < T$? This can happen only if for every $T' < T$, there exists a cluster $D \supsetneq C$ that subsumes C at $G^{T'}$, but does not subsume it anymore at G^T. If D itself was maximal at $G^{T'}$, then the algorithm should have identified D at that time. This gives us an opportunity for large savings in running time. For

every threshold T', after having identified the maximal clusters at $G^{T'}$, we do not need to search the entire network for new maximal clusters at the subsequent threshold, but rather only *within* the maximal clusters of $G^{T'}$. This limits our search space and also enables faster advancement of thresholds.

In practice, our algorithm does not even search within all the maximal clusters, but rather only within the most cohesive ones. Note that the efficiency gains of this approach may come at the price of compromising the ultimate recall guarantee of the algorithm, because we may miss clusters that are subsumed by non-maximal clusters or by noncohesive clusters.

The procedure for identifying candidate clusters in weighted networks is depicted in Figure 6. Given a network G, the procedure sets a threshold T to be the minimum edge weight in G and computes the unweighted network G^T under this threshold. Note that G^T has the same edges as G, except for the minimum weight edges that are eliminated. The procedure then finds the maximal clusters in G^T and adds them to the list of candidate clusters. Next, the procedure recursively searches for more clusters within the clusters of G^T whose cohesion exceeds the *cohesion threshold* β.

The first call to the procedure (i.e., with the original network G) slightly differs from subsequent recursive calls: the threshold T is set to be 0 and not the minimum edge weight. This guarantees that the first unweighted network processed is G^0, which has exactly the same edges as G.

```
1:Procedure weightedCRank(G, β, ℒ)
2:  T := minimum edge weight in G
3:  G^T := unweighted network obtained from G using threshold T
4:  ℒ^T := an empty list of clusters
5:  unweightedCRank(G^T, ℒ^T)
6:  append ℒ^T to ℒ
7:  for all clusters C ∈ ℒ^T for which cohesion(C) ≥ β do
8:     weightedCRank(C, β, ℒ)
```

Figure 6: Identifying candidate clusters in weighted networks.

The recursion stops when reaching a cluster C and a threshold T s.t. C^T cannot be further partitioned into sub-clusters by the procedure that identifies unweighted clusters. This means that C^T must be either a clique or a singleton, and thus C is either a homogeneous clique (i.e., a clique all of whose edges are of the same weight) or a singleton.

Example run Figures 7 and 8 show an example run of the above procedure on a 5-node network G. The procedure applies a threshold of $T = 0$ and obtains the unweighted network G^0 depicted in Figure 7(b). The procedure then finds unweighted clusters in G^0, resulting in the clusters $\{a, b, c, d\}$ and $\{c, d, e\}$ depicted in Figure 7(c). A recursive call is made on each of these two clusters. We focus, for example, on the cluster $\{a, b, c, d\}$ (Figure 8(a)). The minimum edge weight in this cluster is 2, and thus the procedure applies a threshold $T = 2$, resulting in the unweighted network depicted in Figure 8(b). This network breaks into the two clusters $\{a, b, c\}$ and $\{d\}$. More recursive calls are made on these clusters, and we focus on the one made on $\{a, b, c\}$ (Figure 8(c)). The minimum edge weight this time is $T = 5$ and thus the resulting unweighted network is the one depicted in Figure 8(d). Note that the network now consists of singletons only, and therefore the recursion stops. The final list of clusters that will be returned is: $\{a, b, c, d, e\}$, $\{a, b, c, d\}$, $\{c, d, e\}$, $\{a, b, c\}$, $\{a\}$, $\{b\}$, $\{c\}$, $\{d\}$, and $\{e\}$. Some of these clusters (namely, $\{a\}$,$\{b\}$, and $\{e\}$) will be eliminated at the third phase of C-Rank, because they are not maximal.

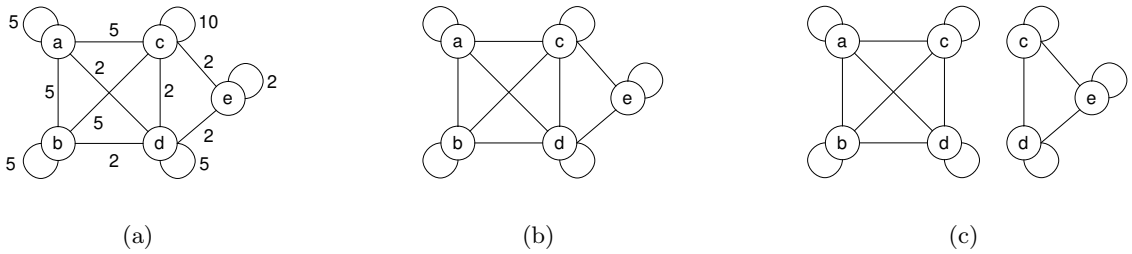


Figure 7: Identifying weighted clusters: Example run.

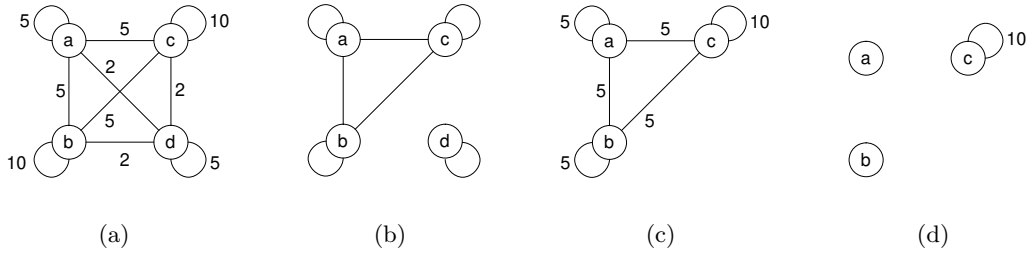


Figure 8: Identifying weighted clusters: Example run (cont).

Setting the threshold The above procedure chooses a threshold T that equals the minimum edge weight in the given network G. If the network has many distinct edge weights and the gaps between them are small, the procedure may need to make many recursive calls, each time eliminating only a small number of edges from the network.

In order to speed up the performance of the procedure on such networks, we select the threshold T differently. Let $w_1, \ldots, w_t$ be the distinct edge weights in G, ordered from smallest to largest. T is set to be the smallest weight that is significantly smaller than the subsequent weight. Formally, let $\gamma > 0$ be the *weight gap parameter* and let i^* be the first $i \in \{1, \ldots, t-1\}$, s.t. $w_i(1+\gamma) \leq w_{i+1}$. T is set to be w_{i^*}.

5.3 Candidate ranking

At its second phase, C-Rank computes the integrated cohesion of each one of the candidate clusters and ranks them accordingly. The main thing to note is that calculating the integrated cohesion of a cluster C requires computing the cohesion of C^T for k values of the threshold T, where k is the number of distinct edge weights in C. Thus, each such calculation requires at most $|E_C|$ sparsest separator calculations, giving a total of $O(|E_C| \cdot r(|C|))$ running time.

5.4 Candidate elimination

The third and last phase of C-Rank consists of eliminating non-maximal clusters from the ranked list of clusters. Testing maximality directly is hard, since to check whether a cluster C is maximal or not, we would need to compare C against all its supersets $D \supsetneq C$. Each comparison entails testing whether D subsumes C under each one of the possible thresholds T. This process requires

exponential enumeration, and moreover every single subsumption test may be prohibitive, since D may have an exponential number of sparsest (or close to sparsest) vertex separators.

Our candidate elimination procedure, therefore, makes two relaxations. First, each cluster C is compared not against all its possible supersets, but rather only against supersets that also belong to the list of candidates. This significantly reduces the search space and makes the enumeration only polynomial in the number of candidates.

Given a candidate cluster D that strictly contains a candidate cluster C, we do not test directly whether D subsumes C under at least one threshold T. We rather simply compare intcohesion(D) with intcohesion(C), and if intcohesion(D)$(1+\epsilon) \geq$ intcohesion(C) (where ϵ is the maximality margin), we declare D as subsuming C. The idea is that if D subsumes C at G^T, then D is at least (and usually more) cohesive than C in G^T. Since cohesion is monotone, D is also expected to be more cohesive than C at $G^{T'}$ for all $T' < T$. This is likely to make the integrated cohesion of D higher (or at least not much lower) than the integrated cohesion of C.

The above is of course a heuristic, and we may thus misclassify some non-maximal clusters as maximal and vice versa, affecting precision and recall. We evaluate the overall recall and precision of the algorithm in Section 7.

6 Finding sparse vertex separators via vertex betweenness

We next address the issue of how to efficiently find sparse vertex separators in unweighted networks. As mentioned above, finding the sparsest vertex separator is NP-hard, yet a separator whose sparsity is at most $O(\sqrt{\log n})$ times the optimum can be found in polynomial time [10]. Nevertheless, the approximation algorithm is based on semi-definite programming, which can be quite inefficient to solve in practice. We therefore opt for a heuristic estimate of the sparsest separator via network flow.

6.1 Relating cohesion and vertex congestion

We start by giving some definitions related to network flow and congestion.

Let $G = (V_G, W_G)$ be a directed, edge-weighted, and strongly connected graph, where $n = |V_G|$. A *uniform demand multicommodity flow (UMFP)* on G is a flow f that routes a unit of commodity from every node $u \in V_G$ to every other node $v \in V_G$. There are no capacity constraints on the flow; that is, an arbitrary amount of total flow (summed over all commodities) can be routed over any edge. Formally, we denote by $\mathcal{P}_{uv}$ the set of all simple paths from u to v in G and $\mathcal{P} = \bigcup_{u \neq v} \mathcal{P}_{uv}$. The flow f is defined as a function from $\mathcal{P}$ to $\mathbb{R}^+$ that satisfies

$$\sum_{p \in \mathcal{P}_{uv}} f(p) = 1, \ \forall u, v \in V_G, \ u \neq v.$$

The *congestion* on a vertex $v \in V_G$ due to flow f, denoted $\mathcal{VC}_f(v)$, is the total amount of flow routed through that vertex. Paths that start or end at v contribute half of their flow to the congestion of v:

$$\mathcal{VC}_f(v) = \sum_{\text{p passes through v}} f(p) + \frac{1}{2} \sum_{\text{p starts or ends at v}} f(p),$$

where the first summation is over paths in which v is an internal node.

Proposition 13. *If $n \geq 2$ and if f is a uniform demand multicommodity flow on G, then for each $v \in V_G$, $\mathcal{VC}_f(v) \geq 1$.*

Proof. Since f is a uniform demand flow, each vertex is the endpoint of paths whose total flow is $2 \cdot (n-1)$ and thus the congestion on each vertex is at least $n - 1 \geq 1$. □

The congestion on an edge $e = (u, v)$ (where $u, v \in V_G$) due to flow f, denoted $\mathcal{EC}_f(e)$, is the total amount of flow routed through that edge, normalized by the weight of the edge, $w(e) = W_G(u, v)$:

$$\mathcal{EC}_f(e) = \frac{\sum_{\text{p passes through e}} f(p)}{w(e)}.$$

The vertex (resp., edge) congestion on the network due to f, denoted $\mathcal{VC}_f(G)$ (resp., $\mathcal{EC}_f(G)$), is the maximum congestion on a vertex (resp., edge) in the network: $\max_{v \in V_G} \mathcal{VC}_f(v)$ (resp., $\max_{e \in W_G} \mathcal{EC}_f(e)$). The *minimum vertex congestion* (resp., *minimum edge congestion*) on the network is defined as the minimum vertex (resp., edge) congestion over all flows: $\mathcal{VC}(G) = \min_f \mathcal{VC}_f(G)$ (resp., $\mathcal{EC}(G) = \min_f \mathcal{EC}_f(G)$).

Remark. The standard way to define a flow is actually different. A flow is defined as a mapping from edges to $\mathbb{R}^+$, rather than from $\mathcal{P}$ to $\mathbb{R}^+$. Moreover, the flow must satisfy the capacity constraints and the value of the flow is defined as the minimum percentage of the demands, satisfied by the flow. It can be shown that the two definitions are equivalent and that the value of the optimal flow essentially equals the inverse of the minimum edge congestion. See Sinclair's work [35] for more details.

Let (U, U^c) be a partition of G into two non-empty, disjoint, and complementary sets. The edge separator, $\langle U, U^c \rangle$, is the set of edges directed from U to U^c. The sparsity of an edge separator in a UMFP is defined as the weight of the separator normalized by the product of the sizes of U and U^c:

$$\rho(U, U^c) = \frac{w(U, U^c)}{|U||U^c|},$$

where $w(U, U^c) = \sum_{e \in \langle U, U^c \rangle} w(e)$ is the *weight* of the separator and the summation is over edges directed from U to U^c. The *sparsest edge separator* of the network, denoted as $\rho(G)$, is the edge separator of minimum sparsity. Leighton and Rao [25] showed the following relationship between the sparsest edge separator and the minimum edge congestion:

Theorem 14 (Leighton-Rao).

$$\frac{1}{\mathcal{EC}(G)} \leq \rho(G) \leq O\left(\frac{\log n}{\mathcal{EC}(G)}\right).$$

Our goal is to show a similar relationship between vertex separators and minimum vertex congestion in undirected and unweighted networks. To this end, we use the following transformation[1], which we denote as φ: given an undirected, unweighted, and connected graph $G = (V_G, E_G)$, we will produce a directed and weighted graph $\varphi(G) = G^* = (V_{G^*}, W_{G^*})$ where

$$V_{G^*} = \{v^{in} | v \in V_G\} \cup \{v^{out} | v \in V_G\}$$

[1] A similar transformation was used by Leighton and Rao [25], however our transformation includes some technical differences, which allow showing the relation between the sparsest vertex separator and the minimum vertex congestion.

and

$$W_{G^*}(v^{in}, v^{out}) = 1, \ \forall v \in V_G,$$
$$W_{G^*}(v^{out}, v^{in}) = \infty, \ \forall v \in V_G,$$
$$W_{G^*}(u^{out}, v^{in}) = \infty, \ \forall (u, v) \in E_G,$$
$$W_{G^*}(v^{out}, u^{in}) = \infty, \ \forall (u, v) \in E_G,$$
$$W_{G^*}(u, v) = 0 \text{ otherwise.}$$

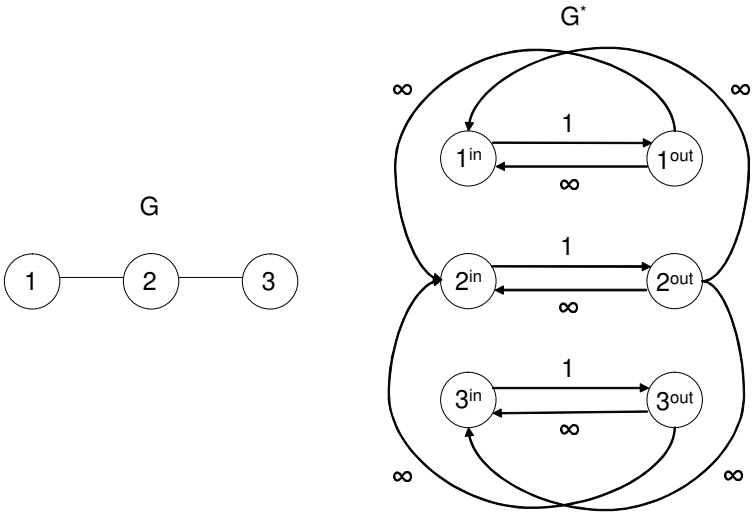


Figure 9: Transforming an unweighted graph to a weighted graph.

Figure 9 demonstrates the operation of φ on a simple graph G.

Lemma 15. *For each UMFP f^* on G^*, there exists a UMFP f on G, such that*

$$\mathcal{VC}_f(G) \leq \mathcal{EC}_{f^*}(G^*).$$

Proof. First, we define $\mathcal{P}^{*,io}$ as the set of all simple paths in G^*, which start at a node u^{in} and end at a node v^{out}, where $u \neq v$

$$\mathcal{P}^{*,io} = \bigcup_{u \neq v \in V_G} \mathcal{P}_{u^{in} v^{out}}.$$

Next, we define $\psi : \mathcal{P}^{*,io} \to \mathcal{P}$ as a function mapping simple paths in G^* that belong to $\mathcal{P}^{*,io}$ to simple paths in G in the following way:

$$\psi(v_1^{in}, v_1^{out}, v_2^{in}, v_2^{out}, ..., v_k^{in}, v_k^{out}) = (v_1, v_2, ..., v_k),$$

where $v_i \in V_G$ and $v_1 \neq v_k$. It is clear by the definition of G^* that each path in $\mathcal{P}$ has exactly one source in $\mathcal{P}^{*,io}$ that is mapped to it by ψ.

Now, given a uniform demand multicommodity flow f^* on G^*, we will construct f as a uniform demand multicommodity flow on G in the following way:

$$\forall p^{*,io} \in \mathcal{P}^{*,io} \text{ s.t. } f^*(p^{*,io}) > 0, \ f(\psi(p^{*,io})) = f^*(p^{*,io}).$$

Note that since (1) f^* passes a unit of commodity in total on all paths between each ordered pair of vertices $s,t \in V_G^*$ s.t. $s \neq t$, and particularly it passes a unit of commodity between each s,t s.t. $s = u^{in}$ and $t = v^{out}$ and (2) each path in $\mathcal{P}$ has exactly one source in $\mathcal{P}^{*'}$ that is mapped to it by ψ , then f must pass a unit of commodity between each pair of vertices $u, v \in V_G$ s.t. $u \neq v$.

From the way f was constructed, it follows that for each $v \in V_G$

$$\begin{aligned}\mathcal{VC}_f(v) \;&\leq \sum_{\text{p passes through v}} f(p) + \sum_{\text{p starts or ends at v}} f(p) \;= \\ &= \sum_{p^{*,io} \in \mathcal{P}^{*,io} \text{ and passes through } (v^{in}, v^{out})} f^*(p^{*,io}) \\ &\leq \; \mathcal{EC}_{f^*}(v^{in}, v^{out}).\end{aligned}$$

As the above inequality holds for each $v \in V_G$, we get the desired result.

□

Corollary 16.

$$\mathcal{VC}(G) \leq \mathcal{EC}(G^*).$$

Lemma 17. *For each UMFP f on G, there exists a UMFP f^* on G^*, such that*

$$\mathcal{EC}_{f^*}(G^*) \leq 5 \cdot \mathcal{VC}_f(G).$$

Proof. We will define four functions $\psi_{ii}, \psi_{io}, \psi_{oi}, \psi_{oo} : \mathcal{P} \to \mathcal{P}^*$ mapping simple paths in G to simple paths in G^* in the following way. Let $p = (v_1, v_2, ..., v_k)$ be a path in G, where $v_i \in V_G$ and $v_1 \neq v_k$. Then,

$$\begin{aligned}\psi_{ii}(p) &= (v_1^{in}, v_1^{out}, v_2^{in}, v_2^{out}, ..., v_k^{in}),\\ \psi_{io}(p) &= (v_1^{in}, v_1^{out}, v_2^{in}, v_2^{out}, ..., v_k^{in}, v_k^{out}),\\ \psi_{oi}(p) &= (v_1^{out}, v_2^{in}, v_2^{out}, ..., v_k^{in}),\\ \psi_{oo}(p) &= (v_1^{out}, v_2^{in}, v_2^{out}, ..., v_k^{in}, v_k^{out}).\end{aligned}$$

In other words, $\psi_{ii}, \psi_{io}, \psi_{oi}, \psi_{oo}$ map each path from u to v in G to four paths in G^* that start at u^{in} or u^{out} and end at v^{in} or v^{out}. It is clear by the definition of G^* that for each $u, v \in V_G$ and for each $x, y \in \{in, out\}$, every path in $\mathcal{P}^*$ from u^x to v^y has exactly one source in $\mathcal{P}$ that is mapped to it by ψ_{xy}.

Now, given a uniform demand multicommodity flow f on G, we will construct f^* as a uniform demand multicommodity flow on G^* in the following way:

$$\begin{aligned}\forall p \in \mathcal{P} \text{ s.t. } f(p) > 0, \quad & f^*(\psi^{ii}(p)) = f^*(\psi^{io}(p)) = f^*(\psi^{oi}(p)) = f^*(\psi^{oo}(p)) = f(p),\\ \forall v \in V_G, \quad & f^*(v^{in}, v^{out}) = 1 \;,\; f^*(v^{out}, v^{in}) = 1,\\ \text{otherwise}, \quad & f^*(p^*) = 0.\end{aligned}$$

Note that from the first rule for constructing f^* and since (1) f passes a unit of commodity in total on all paths between each ordered pair of vertices $u, v \in V_G$ s.t. $u \neq v$ and (2) each path in $\mathcal{P}^*$ from u^x to v^y s.t. $x, y \in \{in, out\}$ and $u \neq v$ has a source in $\mathcal{P}$, it follows that f^* must pass a unit of commodity between each pair of the form (u^x, v^y) s.t. $x, y \in \{in, out\}$ and $u \neq v$. Then, taking into account the second rule for constructing f^*, we get that f^* satisfies all the demands.

As the congestion on all edges whose weight equals ∞ is 0 by definition, the edge congestion on G^* due to f^* derives from the congestion on the edges of the form (v^{in}, v^{out}). From the way f^* was constructed, it can be easily seen that

$$\begin{aligned}\mathcal{EC}_{f^*}(v^{in}, v^{out}) &= \\ &= 4 \cdot \sum_{\text{p passes through v}} f(p) + 2 \cdot \sum_{\text{p starts or ends at v}} f(p) + 1 \\ &= 4 \cdot \mathcal{VC}_f(v) + 1.\end{aligned}$$

Using Proposition 13, we get

$$\mathcal{EC}_{f^*}(v^{in}, v^{out}) \leq 5 \cdot \mathcal{VC}_f(v).$$

Since the above inequality holds for each edge of the form (v^{in}, v^{out}) s.t. $v \in V_G$ and as these are the only edges in G^* whose congestion is greater than 0, we get the desired result. □

Corollary 18.

$$\mathcal{VC}(G) \leq \mathcal{EC}(G^*) \leq 5 \cdot \mathcal{VC}(G).$$

Lemma 19. *G^* is strongly connected.*

Proof. First, it is clear from the definition of G^* that there is a path from v^{in} to v^{out} and from v^{out} to v^{in} for each $v \in V_G$. Then, given an ordered pair of vertices (u^x, v^y), where $x, y \in \{in, out\}$ and $u \neq v$, the path from u^x to v^y will be built based on the path from u to v in G. The latter must exist as G is connected.

□

We showed a tight relation between the minimum vertex congestion in G and the minimum edge congestion in G^*, but in order to relate Theorem 14 to vertex separators, we also need to find the relation between the sparsest vertex separator in G and the sparsest edge separator in G^*.

To this end, we use a slightly different definition for the sparsity of a vertex separator than the one used in Section 4 (we relate the two definitions later in this section). Let S be a vertex separator in G, which induces a partition of $V_G \setminus S$ into A and B. The sparsity of S is

$$\eta_G(S, A, B) = \frac{|S|}{(2|A| + |S|)(2|B| + |S|)}.$$

We now define χ as a function mapping vertex separators in G to finite-weight edge separators in G^* as follows. Let $S = \{s_1, ...s_l\}$ $(l \geq 1)$ be a vertex separator in G, which induces a partition of $V_G \setminus S$ into $A = \{a_1, ..., a_q\}$ $(q \geq 0)$ and $B = \{b_1, ..., b_r\}$ $(r \geq 0)$, where $l + q + r = n$. Then,

$$\chi(S, A, B) = (\{A^{in} \cup A^{out} \cup S^{in}\}, \{S^{out} \cup B^{in} \cup B^{out}\}),$$

where $S^{in} = \{s_1^{in}, ..., s_l^{in}\}$, $S^{out} = \{s_1^{out}, ..., s_l^{out}\}$, and analogously for A^{in}, A^{out}, B^{in}, and B^{out}.

Lemma 20. *The function χ is a one-to-one mapping from the vertex separators in G onto the finite-weight edge separators in G^*.*

Proof. First, we will show that the range of χ consists of finite-weight edge separators only. To this end, we will examine the edges directed from $U = \{A^{in} \cup A^{out} \cup S^{in}\}$ to $U^c = \{S^{out} \cup B^{in} \cup B^{out}\}$. No edges are directed from A^{in} towards U^c as by definition of G^*, all outgoing edges from A^{in} are directed to $A^{out} \subseteq U$. No edges are directed from A^{out} towards U^c as by definition of G^*, such edges would have been possible only towards B^{in}, however as S separates A and B in G, such edges do not exist in G^*. Hence, the only edges from U to U^c are edges from S^{in} and by definition of G^*, such edges can only be directed towards S^{out}. As all edges from S^{in} to S^{out} are of finite weight, the total weight of the separator is finite.

The fact that χ is a one-to-one mapping follows directly from the definition of χ.

Finally, we will show that every finite-weight edge separator in G^* has a vertex separator in G that is mapped to it by χ. Given a finite-weight edge separator $\langle U, U^c \rangle$, we will show a corresponding vertex separator $\{S, A, B\}$ in G. As $\langle U, U^c \rangle$ has finite weight, all edges from U to U^c must be of weight 1, i.e., directed from v^{in} to v^{out}. We will define $S = \{v : (v^{in}, v^{out}) \in \langle U, U^c \rangle\}$. Note that since G^* is strongly connected and since U and U^C are not empty, each edge separator $\langle U, U^c \rangle$ must contain at least one edge directed from U to U^c and thus S is non-empty. As no edges of the form (v^{out}, v^{in}) can be directed from U to U^c, there are no other vertices $u \in V_G \setminus S$ s.t. u^{in} and u^{out} are on opposite sides of the cut (U, U^c). Hence, we can define $A = \{v : \{v^{in}, v^{out}\} \in U\}$ and $B = \{v : \{v^{in}, v^{out}\} \in U^c\}$. Note that $S \cap A = \emptyset$, $S \cap B = \emptyset$, $A \cap B = \emptyset$, and $S \cup A \cup B = V_G$. Note also that no edge can connect a node $u \in A$ with a node $v \in B$, as there would then be an infinite-weighted edge from $u^{out} \in U$ to $v^{in} \in U^c$. Hence, $\{S, A, B\}$ is a vertex separator in G, for which $\chi(S, A, B) = \langle U, U^c \rangle$.

□

Lemma 21.

$$\rho(\chi(S, A, B)) = \eta_G(S, A, B).$$

Proof. In the proof of the previous lemma, we showed that the edges from $U = \{A^{in} \cup A^{out} \cup S^{in}\}$ to $U^c = \{S^{out} \cup B^{in} \cup B^{out}\}$ consist solely of edges from S^{in} to S^{out} . Since the number of edges from S^{in} to S^{out} is $|S|$ and since the weight of each such edge is equal to 1, we get $w(U, U^c) = |S|$. Putting the last equation in the formula for $\rho(U, U^c)$ and considering the fact that $|U| = 2|A| + |S|$ and $|U^c| = 2|B| + |S|$, we get the desired result. □

Lemma 22.

$$\rho(G^*) = \min_{S,A,B} \eta_G(S, A, B).$$

Proof. From the fact that χ is a one-to-one mapping from the vertex separators in G onto the finite-weight edge separators in G^* (Lemma 20), and from the fact that it preserves the value of sparsity (Lemma 21), it follows that the value of the sparsest vertex separator of G is equal to the value of the sparsest edge separator of G^* (which must derive from a finite-weight edge separator). □

Theorem 23.

$$\Theta\left(\frac{1}{\mathcal{VC}(G)}\right) \leq \min_{S,A,B} \eta_G(S, A, B) \leq \Theta\left(\frac{\log n}{\mathcal{VC}(G)}\right).$$

Proof. Corollary 18 ties the minimum vertex congestion in G with the minimum edge congestion in G^*. Then, Lemma 22 ties the sparsest vertex separator of G with the sparsest edge separator of G^*. Using these two relations with Theorem 14 for G^*, we get the desired result. □

As noted above, we used a slightly different definition for the sparsity of vertex separators than the one used in Section 4. We now relate the two definitions:

Lemma 24.

$$\min_{S,A,B} \eta_G(S,A,B) = \Theta\left(\frac{\text{cohesion}(G)}{n}\right).$$

Proof. Feige *et al.* [10] used the following definition for the sparsity of a vertex separator: $\eta'_G(S,A,B) = \frac{|S|}{(|A|+|S|)\cdot(|B|+|S|)}$. This definition is equivalent to the one we use here as

$$\frac{1}{4}\cdot\frac{|S|}{(|A|+|S|)\cdot(|B|+|S|)} = \frac{|S|}{(2|A|+2|S|)(2|B|+2|S|)} \leq$$
$$\leq \frac{|S|}{(2|A|+S)(2|B|+|S|)} \leq \frac{|S|}{(|A|+|S|)(|B|+|S|)}.$$

and thus

$$\eta_G(S,A,B) = \Theta(\eta'_G(S,A,B)) \ , \ \forall\{S,A,B\}.$$

Feige *et al.* show that $\eta'_G(S,A,B) = \Theta\left(\frac{1}{n}\cdot\frac{|S|}{\min\{|A|,|B|\}+|S|}\right)$ (see Feige *et al.* [10], Section 2), and thus we get

$$\eta_G(S,A,B) = \Theta\left(\frac{1}{n}\cdot\frac{|S|}{\min\{|A|,|B|\}+|S|}\right).$$

As this result is valid for any vertex separator $\{S,A,B\}$, we get the desired relationship. □

Corollary 25.

$$\Theta\left(\frac{n}{\mathcal{VC}(G)}\right) \leq \text{cohesion}(G) \leq \Theta\left(\frac{n\cdot\log n}{\mathcal{VC}(G)}\right).$$

In our algorithm (Section 5), we used a normalized version of the congestion on a vertex $v \in V_G$ due to flow f:

$$\mathcal{NC}_f(v) = \frac{1}{\binom{n-1}{2}} \sum_{\text{p passes through v}} f(p).$$

Note that this definition of *normalized congestion* is different from the definition we used so far in two manners: (1) Paths that begin or end with v do not contribute anything to the congestion—as the demands are uniform, each vertex is the endpoint of paths whose total amount of flow is $2(n-1)$. (2) The congestion is normalized by its highest possible value, $\binom{n-1}{2}$, and thus assumes values in $[0,1]$.

$\mathcal{NC}_f(G)$ and $\mathcal{NC}(G)$ are defined analogously to $\mathcal{VC}_f(G)$ and $\mathcal{VC}(G)$.

Lemma 26.

$$\frac{1}{\Theta(1+n\cdot\mathcal{NC}(G))} \leq \text{cohesion}(G) \leq \frac{\log n}{\Theta(1+n\cdot\mathcal{NC}(G))}.$$

Proof. The following relationship between $\mathcal{NC}(G)$ and $\mathcal{VC}(G)$ follows from the definition of normalized congestion:

$$\mathcal{NC}(G) = \frac{\mathcal{VC}(G)-(n-1)}{\binom{n-1}{2}}.$$

The result follows from using the above equation with Corollary 25. □

6.2 Estimating vertex congestion via vertex betweenness

A *shortest path multicommodity flow*, f_{sp}, is one that splits the single unit of commodity routed from u to v evenly among all the shortest paths from u to v. For example, consider the graph depicted in Figure 11(a). There are two shortest paths from vertex a to vertex e: ace and ade. Hence, the shortest path flow, f_{sp}, gives value of $\frac{1}{2}$ to each of these paths. As the graph is undirected, the shortest paths from e to a are, symmetrically, eca and eda. Thus, $f_{sp}(eca) = f_{sp}(eda) = \frac{1}{2}$. In a similar way, as there are two shortest paths between b and e, the value of the flow on the paths bce, bde, ecb, and edb is $\frac{1}{2}$. All other (ordered) pairs of vertices are of adjacent vertices, and thus there is a single shortest path connecting them (the one that goes through the connecting edge). Hence, the flow value of these paths is 1 (e.g: $f_{sp}(ab) = 1$, $f_{sp}(ca) = 1$). The value of f_{sp} for the rest of the paths in the graph is 0.

The *normalized vertex betweenness* of a network G, denoted $\text{NVB}(G)$, is the normalized congestion on G due to the shortest path flow:

$$\text{NVB}(G) = \mathcal{NC}_{f_{sp}}(G).$$

While it is not guaranteed that the shortest path flow achieves the minimum congestion (i.e., $\mathcal{NC}_{f_{sp}}(G) = \mathcal{NC}(G)$), it has been empirically shown to give good estimates of $\mathcal{NC}(G)$ [29]. Furthermore, vertex betweenness can be computed in $O(|V_G||E_G|)$ time using dynamic programming [27].[2] We therefore adopt normalized vertex betweenness as means for estimating the cohesion of networks. Specifically, in our experiments we measure the cohesion of a network G as $1 - \text{NVB}(G)$.

We saw how to use vertex betweenness to measure the cohesion of a network, but how do we use it to find a sparse separator? We employ a simple algorithm (see Figure 10), which computes the NVB *separator* of a network. The algorithm is an adaptation of Girvan and Newman's algorithm [16] for finding edge separators via edge betweenness. Given a network G, the algorithm initializes the separator to the empty set. It then repeatedly executes the following steps, until the network becomes disconnected. The algorithm calculates the NVB of each node in the network, and adds the node v^* with highest NVB to S. When there are several nodes with maximum NVB, v^* is chosen to be the one with smallest id. The algorithm then removes v^* and all its incident edges from the network. Note that the NVB values are re-computed at each iteration of the loop.

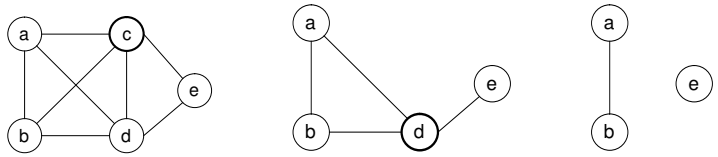
```
1:Function computeNVBSeparator(G)
2:  S := ∅
3:  while G is connected do
4:    calculate NVB(v) for all v ∈ G
5:    v* := node in G of highest NVB
6:    add v* to S
7:    remove v* and all its incident edges from G
8:  return S
```

Figure 10: Algorithm for computing the NVB separator of a network.

Figure 11 shows an example run of the above algorithm on a 5-node network. The nodes c and d have the maximum NVB values ($\frac{1}{6}$) and c is chosen to be removed (Figure 11(a)), since it has a smaller id. After removing c and recalculating NVB values, d turns out to be the node of highest NVB value ($\frac{2}{3}$) and is thus removed (Figure 11(b)). The network is then decomposed into

[2] Newman's algorithm was actually designed to compute another un-normalized version of vertex betweenness. We adapted it to compute normalized vertex betweenness.

two connected components: $\{a, b\}$ and $\{e\}$ (Figure 11(c)), and thus $\{c, d\}$ is the NVB separator of the network.



(a) (b) (c)

Figure 11: Computing an NVB separator: Example run.

As the process of finding the node with highest NVB value might repeat $O(|V_G|)$ times in the worst case, the running time of the above algorithm is $O(|V_G|^2|E_G|)$. Note that this is a heuristic algorithm and thus it is not guaranteed that it finds the sparsest vertex separator.

7 Experiments

7.1 Experimental setup

We tested C-Rank on our own mailboxes as well as on the Enron email data set[3], which consists of 150 mailboxes of Enron employees. The data set contains more than 500,000 messages, mostly sent along the years 2000-2002.

Given a mailbox, we constructed two corresponding networks—an *inbox network* and an *outbox network*—as follows. First, we cleaned the data, by removing duplicate messages, merging alias addresses, and ignoring messages that did not include the mailbox's owner as an explicit sender or recipient. We then split the messages into "outgoing" and "incoming". All the incoming messages were used to construct the inbox network and all the outgoing messages were used to construct the outbox network. The inbox (resp., outbox) network consists of all contacts that appear on headers of incoming (resp., outgoing) messages, excluding the mailbox's owner. Two contacts are connected by an edge if and only if they appear on at least one message header together. The weight of the edge is the number of message headers on which they co-occur. The self loop weight of a contact is the number of message headers on which it appears.

We ran C-Rank with the following parameters: (1) maximality margin $\epsilon = 0.75$; (2) subsumption parameter $\alpha = 0$; (3) cohesion threshold $\beta = 1$; (4) weight gap $\gamma = 0.75$. In most of the experiments, we ignored the self loop weights altogether, in order to focus on the non-singleton communities, which are less trivial to find and rank.

We enforced a hard time limit of 3,600 seconds on the execution of C-Rank on each mailbox. C-Rank was unable to finish its execution on 19 of the 150 mailboxes by this time limit, and thus these mailboxes were excluded from the data set. We ran the experiments on Intel Pentium 4 2.8GHz processor workstations with 2GB of RAM.

Evaluating clustering results automatically is a difficult task. Our situation is even more complicated, because there is no benchmark cluster ranking algorithm to which we could compare C-Rank. We thus created such a benchmark from the widely used edge betweenness hierarchical

[3] http://www.cs.cmu.edu/∼enron.

clustering algorithm of Girvan and Newman [16] (In fact, we used Newman's variant of the algorithm [28], which is adapted to weighted networks). The benchmark algorithm, which we call EB-Rank, is identical to C-Rank, except that it generates its candidate clusters using the edge betweenness algorithm. The ranking and candidate elimination phases of EB-Rank are identical to those of C-Rank.

7.2 Results

Anecdotal results In order to give a feel of the communities produced by C-Rank, we start with some anecdotal results from two of our mailboxes and from a mailbox of an Enron employee. Figure 12 shows the top 10 non-singleton communities in the inbox of Ziv Bar-Yossef. The example demonstrates that the strong communities output by the algorithm are indeed meaningful, as the owner could easily attach a title to each one of them. This list consists of few overlapping communities, since Ziv's research projects tend to be separated and have very few common participants.

Rank	Weight	Size	Member IDs	Description
1	163	2	1,2	grad student + co-advisor
2	41	17	3-19	FOCS program committee
3	39.2	5	20,21,22,23,24	old car pool
4	28.5	6	20,21,22,23,24,25	new car pool
5	28	2	26,27	colleagues
6	28	2	28,29	colleagues
7	25	3	26,30,31	colleagues
8	19	3	32,33,34	department committee
9	15.9	19	35-53	jokes forwarding group
10	15	14	54-67	reading group

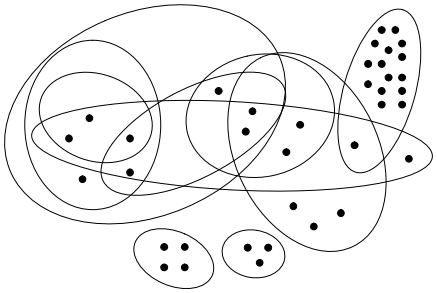
Figure 12: Ziv Bar-Yossef's top 10 non-singleton inbox communities.

Figure 13 shows the top 10 communities output for the inbox of Ido Guy, including singleton communities. This example demonstrates that singleton communities can blend well with non-singleton communities and that they do not necessarily dominate the list of strong communities. In fact, Ido's list is quite diverse in terms of community sizes, ranging from singletons to groups of over 10 participants. The workplace-related communities are highly overlapping, corresponding to different projects with overlapping teams or to different sub-groups within the same project.

Rank	Weight	Size	Member IDs	Description
1	184	2	1,2	project1 core team
2	87	1	3	spouse
3	75	1	4	advisor
4	70.3	4	1,5,6,7	project2 core team
5	62	1	8	former advisor
6	48.2	6	1,2,9,10,11,12	project1 new team
7	46.9	13	13-25	academic course staff
8	46.7	9	1,5,6,7,26-30	project2 extended team (IBM)
9	42.3	5	1,2,9,10,31	project1 old team
10	41.3	13	1,5,6,7,26-30,32-35	project2 extended team (IBM+Lucent)

Figure 13: Ido Guy's top 10 inbox communities.

Figure 14(a) presents a graphical visualization of the top 10 non-singleton communities in the inbox of Eric Bass, an Enron employee. This graph demonstrates that C-Rank is able to reveal quite intricate structures of overlapping communities.



(a) Graphical visualization (non-singleton communities)

Rank	Weight	Member Emails	Description
1	71	shanna.husser@enron.com	spouse
2	60	david.baumbach@enron.com	friend
3	40	luis.mena@enron.com	friend
4	34	brian.hoskins@enron.com hector.campos@enron.com lenine.jeganathan@enron.com	friends
5	31	daphneco64@bigplanet.com daphneco64@yahoo.com jason.bass2@compaq.com lwbthemarine@bigplanet.com	mother, brother, father
6	29	admin.enron@enron.com	Enrom mail sweeper admin
7	25	noreply@ccomad3.uu.commissioner.com	sponsorship bar
8	16	bryan.hull@enron.com chad.landry@enron.com matthew.lenhart@enron.com phillip.love@enron.com timothy.blanchard@enron.com	friends
9	16	brian.hoskins@enron.com hector.campos@enron.com lenine.jeganathan@enron.com luis.mena@enron.com shanna.husser@enron.com	spouse and mutual friends
10	13	bryan.hull@enron.com david.baumbach@enron.com harry.bucalo@enron.com matthew.lenhart@enron.com michael.walters@enron.com o'neal.winfree@enron.com phillip.love@enron.com timothy.blanchard@enron.com	frineds

(b) Detailed description

Figure 14: Eric Bass's top 10 inbox communities.

To complete the picture, Figure 14(b) shows the top 10 communities in the inbox of Eric Bass, including singleton communities. The description of each community was detected by inspecting the content of the messages.

Enron data set statistics Next, we present some statistical data about the results of C-Rank on the 131 mailboxes of the Enron data set. Figure 15 shows the distribution of outbox and inbox network sizes (number of nodes). The networks were ordered by size from smallest to largest, and split into 10 deciles. The height of the bar at each decile represents the median size of networks belonging to this decile. The graph indicates that the mailboxes are of highly variable sizes, while inboxes are significantly larger than outboxes. Some mailboxes at the top decile consist of thousands of nodes each.

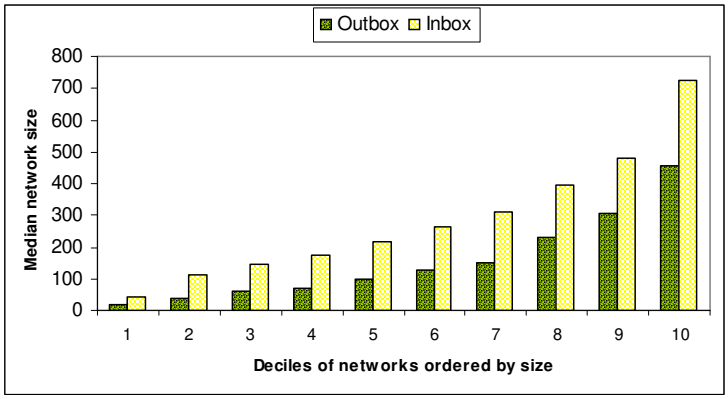


Figure 15: Enron network sizes.

Figure 16 analyzes the running time of C-Rank on the mailboxes on which it finished its execution by the time limit. For each decile of the networks, when ordered by size, the median running time in seconds is given. The graph clearly exhibits the exponential running time behavior of C-Rank. Yet, we note that C-Rank was able to run on networks of 1,075 nodes in about 3 minutes, since the running time depends not only on the network size, but also on the "intricacy" of the community structure of the network and on the number of communities found.

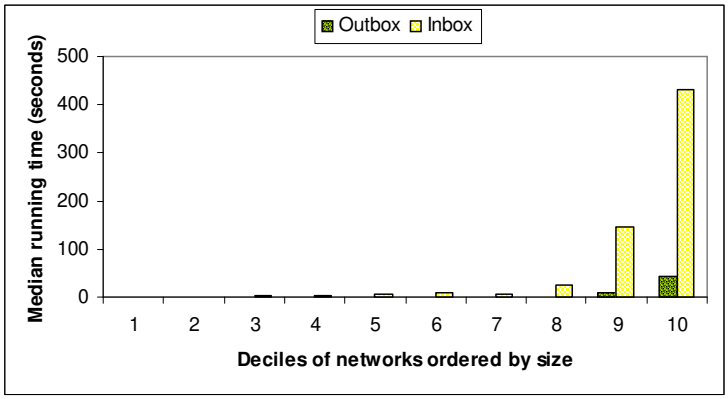
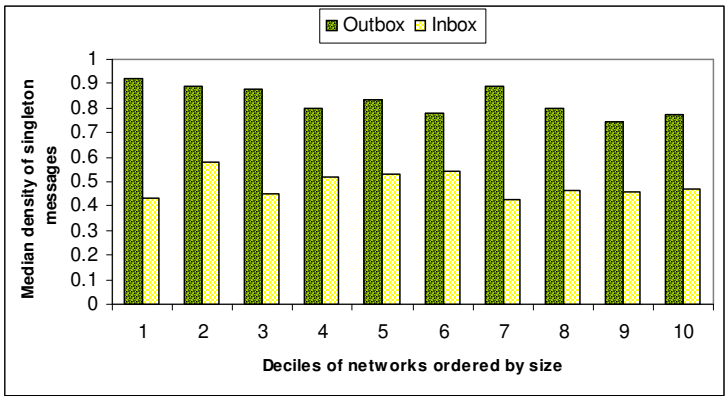


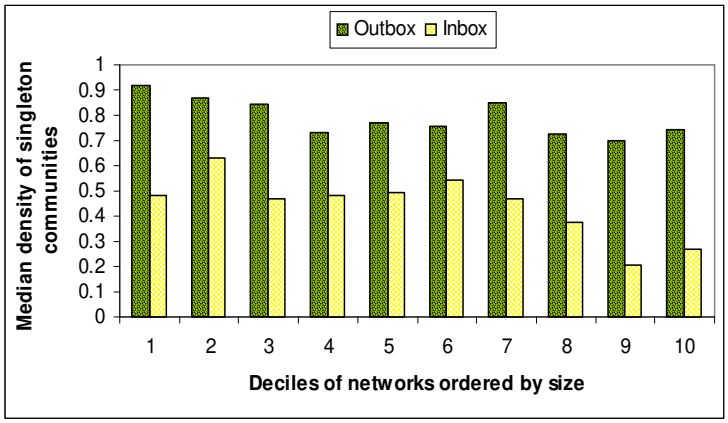
Figure 16: Performance of C-Rank on the Enron data set.

Figure 17 provides evidence of the prominence of "singletons" in the data set. A "singleton message" is one that has only one sender or one recipient, apart from the mailbox's owner. Such a message contributes only to the self loop weight of the corresponding sender/recipient. Figure 17(a) shows that about 80% of the outgoing messages and 50% of the incoming messages, regardless of the mailbox size, were singletons. This huge density of singleton messages necessarily affected

also the results of C-Rank. Indeed, 70% to 90% of the outbox communities and 20% to 65% of the inbox communities detected by C-Rank were singleton communities (Figure 17(b)). We conclude that the high density of singleton communities should be attributed to the nature of the data set, rather than to biases of C-Rank. Since singletons are easy to handle separately, in the rest of our experiments, we eliminated the self loops from the network, and thus focused only on the analysis of non-singleton communities.



(a) Density of singleton messages



(b) Density of singleton communities

Figure 17: Density of singletons in the Enron data set.

Figure 18 depicts the distribution of community sizes output by C-Rank. For each mailbox, we ordered all the output communities by their size, split them into 10 deciles, and calculated the median community size in each decile. We then plotted for each decile the median of these median values, over all mailboxes. The results demonstrate that C-Rank is not biased towards small communities, as one may suspect initially. The median community size at the top decile, for example, was about 20 contacts!

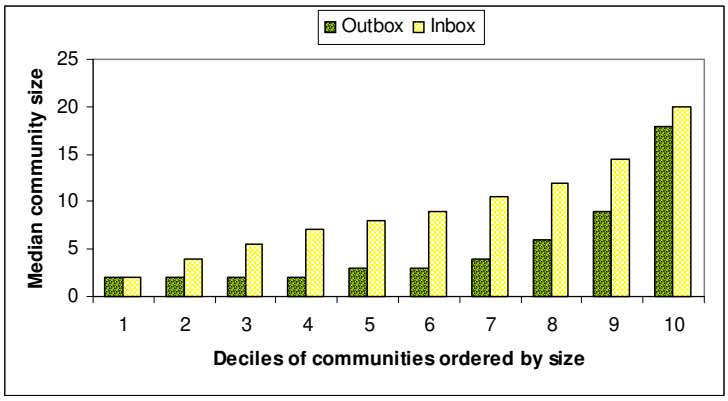


Figure 18: Sizes of communities identified by C-Rank.

Comparison with EB-Rank We now describe a set of experiments that compare the results of C-Rank with the results of EB-Rank (the edge betweenness based algorithm) on the Enron data set. Figure 19 compares the numbers of maximal communities identified by the two algorithms. For each decile of the mailboxes, when ordered by network size, we plotted the median number of maximal communities identified in that decile. The results clearly indicate that C-Rank is able to discover many more maximal communities than EB-Rank. The larger the network size, the more dramatic is the difference. At the top decile, for example, the number of inbox communities found by C-Rank was about 5 times larger than the number of communities identified by EB-Rank. This difference underscores the advantage of overlapping clustering over partitional clustering, at least in this application domain.

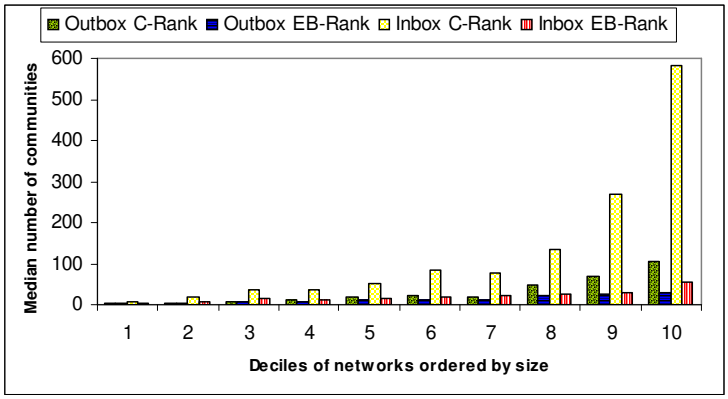


Figure 19: Number of communities identified by C-Rank and EB-Rank.

In Figure 20 we compare the precisions of C-Rank and EB-Rank. Precision was calculated as follows. For each mailbox, we compared the number of communities eventually output by the algorithm (after elimination of non-maximal communities) to the number of communities identified at the candidate identification phase. This ratio was assumed to represent the precision of the algorithm on this mailbox. We then ordered the networks by size, and split them into 10 deciles. We plotted the median precision of the algorithm in each decile. The results shown in the graph demonstrate that precision goes down with network size. The explanation is quite simple: large networks tend to be richer in complex community patterns, and "onions" (see Section 4.3) in

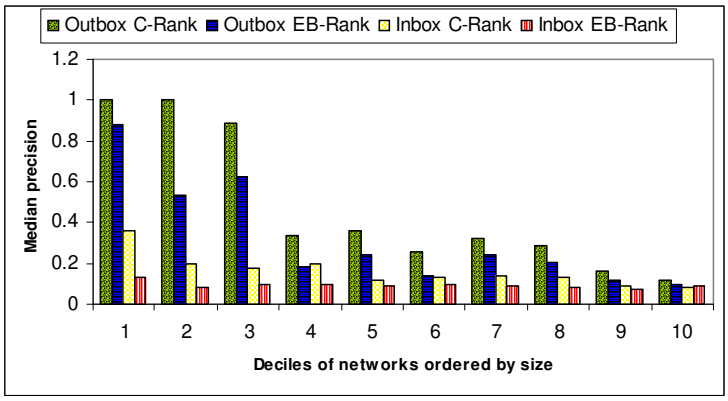


Figure 20: Estimated precision of C-Rank and EB-Rank.

particular. Such patterns give rise to a large number of non-maximal communities, some of which are selected in the first phase of the algorithm. Most of these communities are filtered at the elimination phase of the algorithm. Surprisingly, although C-Rank outputs many more communities than EB-Rank, its precision is comparable and even better than that of EB-Rank.

Figure 21 compares the relative recall of C-Rank and EB-Rank. For each mailbox, we calculated the recall of algorithm A relative to algorithm B as follows. We compared the two lists $\mathcal{L}_A$ and $\mathcal{L}_B$ of communities output by A and by B, respectively, when running on this mailbox. Intuitively, the recall of A relative to B should be the fraction of the communities in $\mathcal{L}_B$ that also appear in $\mathcal{L}_A$. However, even when A and B detect the same community, they may have slightly different "versions" of that community, differing in a few nodes. Therefore, when searching the list $\mathcal{L}_A$ for a community C that shows up on the list $\mathcal{L}_B$, we did not look for an exact copy of C, but rather for a community C' that is "comparable" to C. Formally, we say that C' is *comparable* to C, if $C' \supseteq C$ and $\text{intcohesion}(C')(1+\epsilon) \geq \text{intcohesion}(C)$, where ϵ is the maximality margin. The recall of A relative to B on the specific mailbox was then calculated as the fraction of the communities in $\mathcal{L}_B$, for which we found a comparable community in $\mathcal{L}_A$.

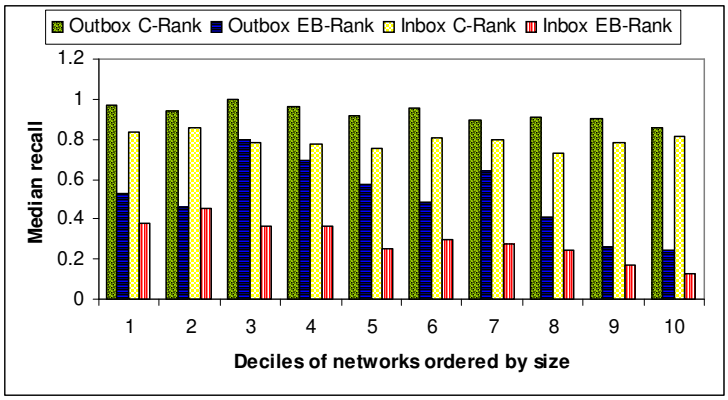


Figure 21: Relative recall of C-Rank and EB-Rank.

After calculating the recall for each mailbox, we ordered the networks by their size, split into 10 deciles, and plotted the median recall at each decile. The results prove that the recall of C-Rank relative to EB-Rank is significantly higher than the recall of EB-Rank relative to C-Rank. The

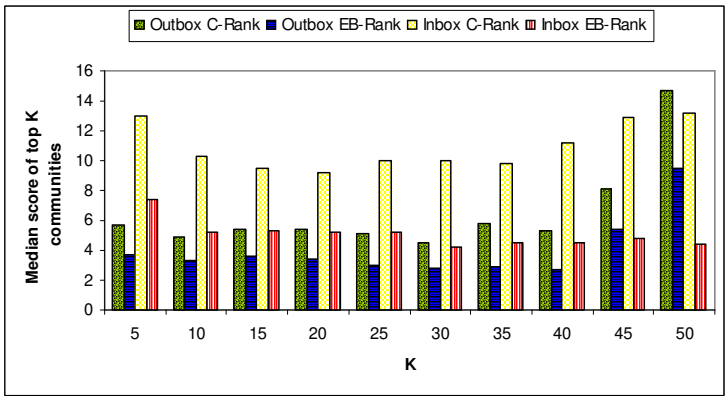


Figure 22: Comparison of median scores of C-Rank and EB-Rank.

difference even becomes higher for larger networks. This experiment stresses again the advantage of using C-Rank, which can detect overlapping communities, to EB-Rank, which is a partitional hierarchical clustering algorithm.

The previous experiments showed that C-Rank is much more successful than EB-Rank in detecting many maximal communities. However, is it possible that these extra communities are all weak, and if we focus only on the strong communities then the two algorithms are comparable? In order to explore this possibility, we compared the strength scores of the communities output by the two algorithms. For each mailbox and for each $k = 5, 10, 15, \ldots, m$, where m is the minimum number of communities output by the two algorithms on this mailbox, we calculated the median integrated cohesion of the top k communities on each of the two output lists. For each k and for each algorithm, we then plotted the median score over all networks for which $m \geq k$ (see Figure 22). These results indicate that C-Rank not only finds more maximal communities overall, but also finds *better* communities. This phenomenon is consistent across inbox and outbox and across different values of k.

Robustness experiments One indication of a good clustering algorithm is that it is robust to small changes in the data. In order to test the robustness of C-Rank, we compared the communities it output when running over on the entire Enron data set to the communities it output when running over a sample of the data. For this experiment, we focused only on sufficiently large mailboxes: ones in which the number of messages was at least 500. Overall, we used 36 outboxes and 41 inboxes in this experiment. For each such mailbox, we constructed 3 networks: one that was constructed using all the messages in the mailbox, one that was constructed using 80% randomly chosen messages from the mailbox, and one that was constructed using 20% randomly chosen messages from the mailbox. (The latter two networks were constructed 5 times each, and the results presented here are the medians over these 5 trials.) For each of the two latter networks, and for each $p = 10\%, 20\%, \ldots, 100\%$, we calculated the recall of the top $k = p \cdot m$ communities output by C-Rank on this network (where m is the total number of communities output on this network) relative to the top k communities output by C-Rank when running on the first, complete, network. For each p, we then calculated the median recall over all networks. This value, which we call "recall@p", captures how well C-Rank was able to detect the strong communities of the mailbox, when running over only a portion of the data in the mailbox.

The results indicate that C-Rank is rather resilient to random removal of data. On the networks built over 80% of the data, C-Rank was able to maintain a recall of about 90% across all values of

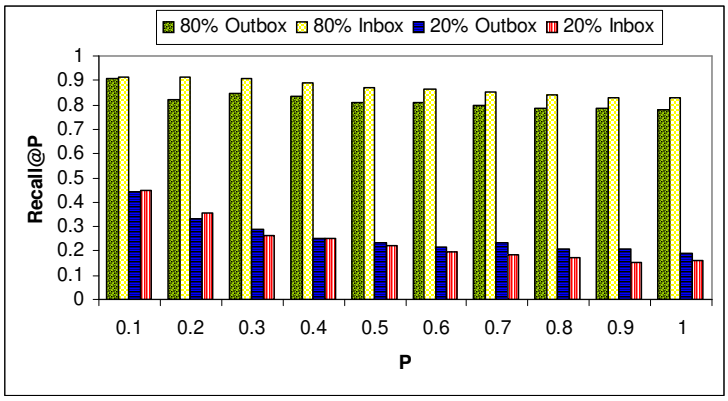


Figure 23: Relative recall of C-Rank on samples from the Enron data set.

p. When running on a mere 20% of the data, C-Rank was still able to maintain reasonable recall of 45% at the top decile and 20% at the bottom decile.

To sum up, we believe that the above experiments provide convincing evidence that C-Rank is able to achieve high recall values (i.e., covering many of the maximal clusters in the network), while maintaining a relatively high precision. C-Rank is completely superior to EB-Rank, which is based on the very popular edge betweenness clustering algorithm. C-Rank is also robust to random removal of data, attesting to its quality.

8 Conclusions

We presented the cluster ranking problem as a novel framework for clustering. We then proposed integrated cohesion as a new strength measure for clusters. We designed C-Rank: a cluster ranking algorithm that detects and ranks overlapping clusters in arbitrary weighted networks. We demonstrated the effectiveness of the framework and the algorithm by ranking clusters in egocentric mailbox networks.

Finding a sparse vertex separator is an important part of C-Rank. To this end, we proposed a heuristic algorithm, which is based on a new connection between vertex betweenness and multi-commodity flow.

An interesting problem following this work is whether one can design an efficient cluster ranking algorithm, which given an integer k, finds the *top* k maximal clusters directly, without computing the full list of maximal clusters. Such an algorithm could be much more efficient than C-Rank, when the list of maximal clusters is very long. We believe that finding the top k clusters exactly is NP-hard, yet finding some "good" k clusters may be feasible to do. Addressing this question is left for future work.

Acknowledgments

We thank Gail Gilboa-Freedman, Nili Ifergan, Idit Keidar, Natalia Marmasse, Elad Shahar, Uzi Shvadron, Eyal Sonsino, and Gala Yadgar for sharing with us their mailboxes. We thank Ran El-Yaniv and Ran Wolff for very helpful suggestions.

References

[1] E. Amir, R. Krauthgamer, and S. Rao. Constant factor approximation of vertex-cuts in planar graphs. In *Proceedings of the 35th ACM Symposium on Theory of Computing (STOC)*, pages 90–99, 2003.

[2] A. Banerjee, C. Krumpelman, J. Ghosh, S. Basu, and R. J. Mooney. Model-based overlapping clustering. In *Proceedings of the 11th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 532–537, 2005.

[3] J. D. Banfield and A. E. Raftery. Model-based guassian and non-gaussian clustering. *Biometrics*, 49:803–821, 1993.

[4] J. Baumes, M. K. Goldberg, M. S. Krishnamoorthy, M. Magdon-Ismail, and N. Preston. Finding communities by clustering a graph into overlapping subgraphs. In *Proceedings of the IADIS International Conference on Applied Computing*, pages 97–104, 2005.

[5] J. Baumes, M. K. Goldberg, and M. Magdon-Ismail. Efficient identification of overlapping communities. In *Proceedings of the IEEE International Conference on Intelligence and Security Informatics (ISI)*, pages 27–36, 2005.

[6] P. O. Boykin and V. Roychowdhury. Personal email networks: An effective anti-spam tool. *IEEE Computer*, 38(4):61–68, 2005.

[7] T. N. Bui and C. Jones. Finding good approximate vertex and edge partitions is NP-hard. *Information Processing Letters*, 42:153–159, 1992.

[8] G. Cleuziou, L. Martin, and C. Vrain. PoBOC: An overlapping clustering algorithm, application to rule-based classification and textual data. In *Proceedings of the 16th Eureopean Conference on Artificial Intelligence (ECAI)*, pages 440–444, 2004.

[9] D. Fasulo. An analysis of recent work on clustering algorithms. Technical Report 01-03-02, Department of Computer Science and Engineering, University of Washington, Seattle, 1999.

[10] U. Feige, MT. Hajiaghayi, and J. R. Lee. Improved approximation algorithms for minimum-weight vertex separators. In *Proceedings of the 37th ACM Symposium on Theory of Computing (STOC)*, pages 563–572, 2005.

[11] D. Fisher. Using egocentric networks to understand communication. *IEEE Internet Computing*, 9(5):20–28, 2005.

[12] D. Fisher and P. Dourish. Social and temporal structures in everyday collaboration. In *Proceedings of the 2004 Conference on Human Factors in Computing Systems (CHI)*, pages 551–558, 2004.

[13] G. W. Flake, S. Lawrence, and C. L. Giles. Efficient identification of Web communities. In *Proceedings of the 6th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 150–160, 2000.

[14] G. W. Flake, S. Lawrence, C. L. Giles, and F. Coetzee. Self-organization and identification of web communities. *IEEE Computer*, 35(3):66–71, 2002.

[15] L. C. Freeman. *The Development of Social Network Analysis: A study in the Sociology of Science*. Empirical Press, Vancouver, 2004.

[16] M. Girvans and M. E. J. Newman. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences of the United States of America (PNAS)*, 99(12):7821–7826, 2002.

[17] F. Höppner, F. Klawonn, R. Kruse, and T. Runkler. *Fuzzy Cluster Analysis: Methods for Classification, Data Analysis and Image Recognition*. Wiley & Sons, 1999.

[18] H. Ino, M. Kudo, and A. Nakamura. Partitioning of Web graphs by community topology. In *Proceedings of the 14th international conference on World Wide Web (WWW)*, pages 661–669, 2005.

[19] A. K. Jain and R. C. Dubes. *Algorithms for Clustering Data*. Prentice-Hall, 1988.

[20] A. K. Jain, A. P. Topchy, M. H. C. Law, and J. M. Buhmann. Landscape of clustering algorithms. In *Proceedings of the 17th International Conference on Pattern Recognition (ICPR)*, volume 1, pages 260–263, 2004.

[21] R. Kannan, S. Vempala, and A. Vetta. On clusterings: Good, bad and spectral. *Journal of the ACM*, 51(3):497–515, 2004.

[22] J. M. Kleinberg. An impossibility theorem for clustering. In *Proceedings of the 15th Annual Conference on Neural Information Processing Systems (NIPS)*, pages 446–453, 2002.

[23] B. Klimt and Y. Yang. The enron corpus: A new dataset for email classification research. In *Proceedings of the 15th European Conference on Machine Learning (ECML)*, pages 217–226, 2004.

[24] R. Kumar, P. Raghavan, S. Rajagopalan, and A. Tomkins. Trawling the web for emerging cyber-communities. In *Proceedings of the 8th International World-Wide Web Conference (WWW)*, pages 1481–1493, 1999.

[25] T. Leighton and S. Rao. Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms. *Journal of the ACM*, 46(6):787–832, 1999.

[26] A. McCallum, A. Corrada-Emmanuel, and X. Wang. Topic and role discovery in social networks. In *Proceedings of the 19th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 786–791, 2005.

[27] M. E. J. Newman. Scientific collaboration networks: II. Shortest paths, weighted networks, and centrality. *Phyical Review E*, 64(016132), 2001.

[28] M. E. J. Newman. Analysis of weighted networks. *Phyical Review E*, 70(056131), 2004.

[29] M. E. J. Newman and M. Girvans. Finding and evaluating community structure in networks. *Phyical Review E*, 69(026113), 2004.

[30] G. Palla, I. Derényi, I. Farkas, and T. Vicsek. Uncovering the overlapping community structure of complex networks in nature and society. *Nature*, 435:814–818, 2005.

[31] F. C. N. Pereira, N. Tishby, and L. Lee. Distributional clustering of english words. In *Proceedings of the 31st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 183–190, 1993.

[32] J. Scott. *Social Network Analysis: A Handbook*. Sage, London, 1991.

[33] E. Segal, A. Battle, and D. Koller. Decomposing gene expression into cellular processes. In *Proceedings of the 8th Pacific Symposium on Biocomputing (PSB)*, pages 89–100, 2003.

[34] J. Shi and J. Malik. Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(8):888–905, 2000.

[35] A. Sinclair. Improved bounds for mixing rates of Markov chains and multicommodity flow. *Combinatorics, Probability & Computing*, 1:351–370, 1992.

[36] N. Slonim. *The Information Bottleneck: Theory and Applications*. PhD thesis, The Hebrew University of Jerusalem, 2002.

[37] J. Tyler, D. Wilkinson, and B. A. Huberman. Email as spectroscopy: Automated discovery of community structure within organizations. In *Proceedings of the 1st International Conference on Communities and Technologies*, pages 81–96, 2003.

[38] S. M. van Dongen. *Graph Clustering by Flow Simulation*. PhD thesis, University of Utrecht, 2000.

[39] B. Wellman. An egocentric network tale. *Social Networks*, 15:423–436, 1993.